

Response Letter

Paper 498 — *The Case for Text-to-SQL Friendly Logical Database Design*

We thank the meta-reviewer and the reviewers for the opportunity to improve this paper and for the detailed and constructive comments. We have substantially revised the submission by comparing each proposed operator with corresponding prior work, adding more baselines and datasets, adding more in-depth discussion on robustness and error-analysis, and reframing our findings and contributions.

Changes in the revised paper are highlighted as follows:

- **blue** for changes addressing comments of Reviewer #1,
- **green** for changes addressing comments of Reviewer #2,
- **magenta** for changes addressing comments of Reviewer #5,
- **red** for general changes, or changes addressing comments from more than one reviewer.

A note on numbering. CMT lists the reviews as #1, #2 and #5, while the meta-review refers to them as R1, R2 and R3; the meta-review’s “R3” is CMT’s Reviewer #5. This letter follows the CMT numbering. Reviewer #2’s detailed comments are unnumbered in CMT; per meta-review item #5, we number them D1–D15 in the order they appear.

RESPONSE TO THE META-REVIEWER

Meta-Reviewer Summary: ...*On the positive, the reviewers agreed that ideas in the draft were well described, and brings together several schema-level ideas that that could inspire lots of interesting work in the area. But the concerns are significant: the novelty over prior work that described and even experimentally analyzed similar ideas is not clear and the current version oversells its conceptual contributions. All 3 key innovations that the paper presents (+A abstraction, +P partitioning, and +R renaming) have been described previously in the literature. ... Thus, the revised paper must very clearly separate: what was already known before this paper, what remained unsolved, what is really that the paper newly solves, and also how the empirical results support all that.*

Ans: We thank the meta-reviewer for the opportunity to revise the paper. Here we respond to the common concerns about the novelty over prior work. We agree with the reviewers that renaming, semantic views, and schema pruning are not new operators individually, and we have adjusted our claims in the revised paper. SNAILS [27] studies column identifier naturalness, semantic layers [10] expose denormalized relations, and modern Text-to-SQL systems often perform schema linking and pruning in their pipelines [40].

The question we study is different: can a model-facing logical design be derived automatically from a historical Text-to-SQL workload, and under what conditions does changing that logical representation improve downstream generation? In other words, we move the optimization variable from only the Text-to-SQL pipeline to the logical representation that the pipeline consumes.

Our contribution is therefore threefold. First, we formulate Text-to-SQL friendliness as an explicit logical-design objective. Second, we propose a workload-driven design procedure in which recurring access patterns determine which abstractions and partitions are exposed, while the underlying schema remains unchanged. Third, we empirically separate this workload-driven design from the closest operator-specific alternatives: SNAILS [27] for renaming, ASR [32]/semantic views and schema-derived star joins for abstraction [10], and TEMPLAR [3]/schema linking [40] plus the schema-vs-demonstration ablation for partitioning.

The revised paper therefore does not claim novelty for views, renaming, or pruning. We claim that historical Text-to-SQL workloads can be treated as a database-design signal for constructing a persistent LLM-facing logical layer, and that the usefulness of such a layer can be measured independently of the downstream model. We have updated the Introduction in the revised paper to reflect prior work, the remaining gap, and reframed our contribution. Figure 1 summarizes the positioning of this work in a way that prior work either only redesigns the logical schema, uses the workload without redesigning the logical schema, or neither.

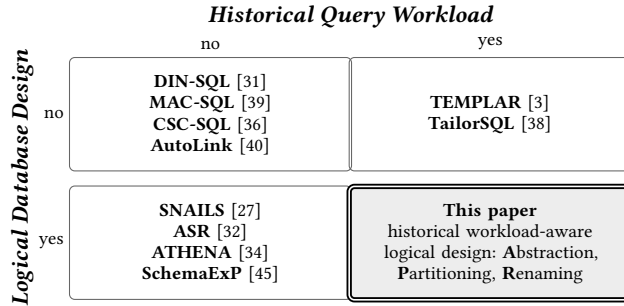


Figure 1: Positioning of prior work by whether it uses historical workloads and redesigns the model-facing logical schema.

Required revision items:*Meta-Reviewer: items 1–5**Reviewer 1: W1–W3, D1–D5**Reviewer 2: W1–W4, D1–D15**Reviewer 5: W1–W3, D3–D5.*

Ans: We have addressed every listed item. The main changes are summarized here; detailed responses with section pointers follow in the itemized responses.

- **Comparisons with prior methods.** We implemented relevant baselines for +R schema renaming: SNAILS [27] and EvoSchema [44], +A views: Agentic Schema Refinement [32], TEMPLAR [3] on the merged schemas and compared them against their respective operators. Our operators outperforms the baselines in most of the evaluated cases (see Figure 8). For +A baseline ATHENA [34] and +P baseline SchemaEXP [45], we added the discussion in Section 2 about why they cannot be applied in a straightforward way (**MR item 1, R1.W1**).
- **A stronger agentic baseline.** We added AutoLink [40] as a backbone pipeline and ran it under every configuration on both Spider-Union and BIRD-Union (in the updated Table 2). The Execution Accuracy for AutoLink increases by 2.7% to 5.7% on BIRD-Union (Section 6.2) (**R1.W3, R1.D5**).
- **A new enterprise benchmark.** We also ran our methods on BEAVER [7] benchmark. Due to the resource limitation, we sampled 1,536 test questions from the three warehouses in BEAVER (Section 6.9)(**R1.W3**).
- **More comparison and discussion on schema normalization.** We also evaluated text-to-SQL pipelines on an FK-derived star schema, a filtered set of views with only three or more tables, and a full denormalization schema. (**R2.W1, R2.D3, R5.W1**).
- **Result presentation.** We added McNemar’s exact test to the main experiment result at two thresholds ($p < 0.1$ and $p < 0.05$); for +A + P + R configuration, we added the standard deviation from three independent runs (Section 6.8) (**MR item 3, R1.W3, R1.D2**).
- **Renaming Validation and robustness.** Audit and embedding distance calculation of the renames (Section 6.6), external knowledge and template-disjoint tests (Section 6.7), and an error-propagation check across operators (Section 6.11) (**R2.W3, R1.D1, R1.D4, R5.W2**).
- **Contribution claims.** We rewrote the corresponding paragraphs in the introduction and related work sections to reframe our contribution.
- **Appendix.** Results removed from the paper for space, together with the full version of the tables in this letter, are provided in the appendix in our repository (<https://anonymous.4open.science/r/Logical-Database-Design-CC0C>).

Because the new experiments and comparisons are substantial, we could not keep all results in the main paper. We decided to keep only the core results in the main paper after a heavy reorganization and provide the full result tables in this letter. The main changes to the paper’s structure are:

- Previous Section 6 is reorganized so that each new subsection (Section 6.4, 6.5, 6.6, 6.8, 6.9, 6.10) presents the added comparison with prior methods or error/cost analysis.
- Previous Tables 2–4 are merged into one Table 2 with new AutoLink results; we also added significance marks on each cell.
- Previous Section 6.5 (Per-Domain Analysis and Boundary Conditions), with its SQL examples and previous Table 5 (database statistics), is removed. Moreover, we moved the boundary-condition part to Section 6.5 and 6.11.
- Previous Figures 7–10 (the 0/50/100% history comparison, per-database accuracy, and the join/difficulty breakdown) are removed. Table 2 and Section 6.7 now carry the workload-availability result; the full results are now in the supplementary material.
- Previous Figure 11 (error analysis) is replaced by Figure 10, which scores two pipelines on the questions they gain and lose.

MR item #1 (overstatement of contributions). *Compare experimentally against SNAILS; against SchemaEXP [45], or give a generally accepted reason why that is not possible; against ATHENA and Agentic Schema Refinement for +A; against “Bridging the Semantic Gap” and schema-linking/retrieval approaches for +P. Separate the effect of schema pruning from showing fewer example question–SQL pairs. Either reposition more modestly, or provide much stronger conceptual novelty and/or experimental comparison.*

Ans: Each named method is now compared in the same setting, BASESQL on BIRD-Union [26] and Spider-Union [43], using the prior method to substitute its respective operator. The results are in the new Section 6.4. Here we summarize their performance:

- **SNAILS [27] (for +R)** drops below the baseline in all six cases (three backbones $\times$ two datasets), ranging from -0.6% to -11.9% , whereas +R is better than the baseline in all six cases. SNAILS leaves 73 of 75 tables and 534 of 798 columns unchanged, and the SNAILS-generated column names may be semantically irrelevant, not merely imprecise.
- **Agentic Schema Refinement(ASR) [32] (for +A)** underperforms the baseline slightly in all six cases, by -0.1% to -1.0% . It generates 825 views against our 64, and the average views injected per question for ASR is 35.8 while it is only 4.4 for +A. At the same time, it lengthens the average tokens needed per question on BASESQL by over 20% than baseline and 9% more than +A.

- **EvoSchema [44] (for +A)** merges tables into wider ones, which we find hurts performance by 4.4% on BIRD-Union (Section 6.4).
- **ATHENA [34, 35] (for +A)** is not open-sourced. We used a publicly available repo ¹ reconstructed from the paper. It answers only 8.1% of BIRD-Union questions and 8.5% with our views added, because even with the history, 59.7% of the gold queries need constructs outside its ontology query language. This method requires a domain expert to constantly build the ontology to be useful, and we report this here rather than in the main paper.
- **TEMPLAR [3] (for +P, “Bridging the Semantic Gap” paper)** is slightly positive to slightly negative ranging from -1.7% to $+1.0\%$. It maps keywords to schema elements and infers joins, which leads to a low gold table recall: on the 75-table schema of BIRD-Union, TEMPLAR’s output contains every gold table only 46.2% of the time, and 67.5% of its paths join tables from different databases, whereas +P’s partitions contains the gold table 96.9% of the time while averaging only 5.3 tables.
- **SchemaExp [45] (for +R and +P)** targets TableQA, especially questions involving deriving calculations over columns in single Wikipedia tables (its SQUALL dataset), where one cell often packs several values into a string, and this kind of transformation does not apply to our Text-to-SQL benchmarks. For example, its expansion component is designed for columns that can split into new sub-fields to answer the question (e.g., a Term column with values like 1926–1927 for Term_Start and Term_End), which is specialized to the TableQA benchmarks, while BIRD and Spider datasets do not involve this kind of question. Moreover, the expansion component is rule-based, in which each column has its own label (e.g., year duration), and the expander expands the column based on that label (e.g., derive the Term_Duration from the value 1926–1927 by extracting the two years and performing the year subtraction). Building such rules targets a different goal than our +R operator, which can be an interesting but orthogonal direction. The pruner component is a classifier trained on in-domain data on a single table, so 1) its pruning process only involves pruning columns in a given single table, which is a much simpler case compared to BIRD-Union and Spider-Union; 2) its training process does not directly apply to a general text-to-SQL task, since it requires the relevant table to be provided.
- **Schema-linking/retrieval approaches for +P:** most recent agentic Text-to-SQL pipelines involve a schema linking/retrieval component. For example, AutoLink first shortlists a small candidate column set by matching the question’s words to table and column names, then expands that list over several rounds by probing the database contents and following foreign-key edges, and finally passes the final shortlisted schema for SQL generation. Our +P operator, rather than replacing the schema-linking component, adds a step that narrows the candidate table and column list based on table access in the historical query workload. Therefore, to demonstrate the effectiveness of the +P operator, in the AutoLink pipeline, we integrated +P before its database content probing step, which improves the accuracy from 51.5% to 56.3% on BIRD-Union (See Table 2 in the revised paper).
- **Pruning vs. demonstrations:** To separate the effect of schema pruning from a limited pool of examples, we added the controlled experiment on the combinations of providing full/pruned schema and example pairs retrieved on the full/pruned schema in the prompt (Section 6.3). Here we repeat the result in Table 2 below. On BIRD-Union, pruning the schema alone (Pruned schema $\times$ Global top-k) improves the performance by up to 1.30% compared to the original pipeline (full schema $\times$ global top-k), while providing pruned schema retrieval only shows less improvement. Applying both (Pruned schema $\times$ partition-scoped top-k) together gives +P’s 2.61% performance gain with Gemini 2.5-flash-lite as the backbone.

Table 1: [MR-1, R1.W1, R5.W3] New in § 6.4. Each prior method substituted for the operator it anticipates, under BASESQL with top-k=3; EX with delta over the row’s baseline. **: McNemar $p < 0.05$; *: $0.05 \leq p < 0.10$.

Backbone	Baseline	renaming		abstraction		partitioning	
		+R	SNAILS	+A	ASR	+P	TEMPLAR
<i>BIRD-Union (n=767)</i>							
GPT-4.1 mini	49.80	50.46 (+0.65)	49.02 (-0.78)	50.85 (+1.04)	49.67 (-0.13)	50.46 (+0.65)	50.20 (+0.39)
Gemini 2.5-flash-lite	48.11	48.63 (+0.52)	36.25 (-11.86)	47.98 (-0.13)	47.20 (-0.91)	50.72 (+2.61)**	46.41 (-1.69)
GPT-5.4 mini	52.80	52.93 (+0.13)	49.80 (-3.00)	55.54 (+2.74)**	52.54 (-0.26)	53.06 (+0.26)	52.54 (-0.26)
<i>Spider-Union (n=502)</i>							
GPT-4.1 mini	79.48	80.08 (+0.60)	78.29 (-1.20)	81.27 (+1.79)	79.28 (-0.20)	79.48 (0.00)	80.48 (+1.00)
Gemini 2.5-flash-lite	80.48	81.47 (+1.00)	78.09 (-2.39)	80.48 (0.00)	80.08 (-0.40)	81.67 (+1.20)	80.08 (-0.40)
GPT-5.4 mini	81.67	82.47 (+0.80)	81.08 (-0.60)	84.46 (+2.79)**	80.68 (-1.00)	82.47 (+0.80)	81.87 (+0.20)

¹<https://github.com/postech-db-lab-starlab/ATHENA>

Table 2: [MR-1, R5.D4] New in § 6.3 (Table 3). +P decomposed into schema pruning (rows) and demonstration scoping (columns), BIRD-Union. Global top-k means the few-shot demonstrations are retrieved from all available queries in the workload, while partition-scoped top-k restricts the retrieval to queries that only access the tables/columns in the pruned schema. ** indicates McNemar’s Test result $p < 0.05$.

	Global top-k	Partition-scoped top-k
<i>GPT-4.1 mini</i>		
Full schema	49.80	50.33 (+0.53)
Pruned schema	50.98 (+1.18)	50.46 (+0.66)
<i>Gemini 2.5-flash-lite</i>		
Full schema	48.11	48.11 (+0.00)
Pruned schema	49.41 (+1.30)	50.72 (+2.61)**

MR item #2. Please report more details about your methods, and what is really going on during the transformations (see R3.W1/R3.W2/R3.D3/R3.D4/R2.D3/R2.D4/R2.D9/R2.D10/R2.D11).

Ans: Thanks for clearly listing the required items. We have reported the details in the paper and in the responses to the corresponding reviewer comments.

MR item #3. Improve reporting of the experiments: accuracy claims, effects from your methodology that may be responsible for some of your results, reporting of significance, and more details.

Ans: In the revised main-results tables (e.g., Table 2), we now report McNemar’s exact test for every configuration against the corresponding baseline in the same row, marking ** for $p < 0.05$ and * for $0.05 \leq p < 0.10$. Across the 112 paired comparisons, 25 have $p < 0.05$, with another 16 between 0.05 and 0.10. McNemar’s test is appropriate for this paired setting because both systems are evaluated on the same questions. The test uses only questions on which their correctness differs, comparing the number of baseline-wrong/tested-method-correct cases with the number of baseline-correct/tested-method-wrong cases [12, 13, 29]. Due to space limitations, the revised paper reports the significance level in the main tables; the complete p -values and corresponding 95% confidence intervals are provided in the supplementary material.

Repeated runs. Due to computational and time constraints, we repeat the complete +A+P+R configuration three times for every backbone and benchmark rather than rerunning every ablation. Across these settings, the run-to-run standard deviation is 0.2–0.8 EX percentage points (See Table 6 below).

Efficiency. We additionally report offline preparation cost, per-configuration prompt-token cost, and BEAVER prompt cost in Section 6.10 and our response to **R1.D3**.

Repeated runs. Due to the resource and time limitation, we report three runs of the full +A+P+R combination on every backbone and benchmark. The standard deviation ranges from 0.2% to 0.8% (Table 6).

Efficiency. Offline cost, prompt tokens per configuration, and BEAVER prompt cost are reported in Section 6.10 and under R1.D3.

MR item #4. Add more baselines and comparisons to your experiments.

Ans: We have added **AutoLink** [40] as a strong recent baseline. AutoLink is an agentic Text-to-SQL pipeline that uses DeepSeek-V3 for schema linking and DeepSeek-R1 for SQL generation. We evaluate AutoLink under all seven transformation configurations on both BIRD-Union and Spider-Union; we report the results in Section 6.2 in the paper and repeat the summary in Table 3 below. On BIRD-Union, every transformation configuration improves over the AutoLink baseline, with gains ranging from +2.74 to +5.74 EX percentage points; the best configuration, +A+R, improves EX from 51.50% to 57.24%. On Spider-Union, all seven configurations also improve over the AutoLink baseline.

We also broaden the evaluation to **BEAVER**, an enterprise data-warehouse benchmark containing three databases. Due to computational and time constraints, we sample 1,534 historical queries and 1,536 evaluation questions across the three databases. Section 6.9 in the revised paper reports the aggregate results, while Table 4 below reports results separately for each database. BEAVER is substantially more challenging for the underlying Text-to-SQL pipeline: BASESQL achieves 21.3% EX. Nevertheless, our transformations improve performance across all three BEAVER databases, providing additional evidence that the observed gains are not specific to BIRD-Union or Spider-Union.

Table 3: [R1.W3/D5] New in § 6.2 (AutoLink rows of Table 2). AutoLink (DeepSeek-V3/R1) under every configuration. ** $p < 0.05$.

	Base	+A	+P	+R	+A+P	+A+R	+P+R	+A+P+R
BIRD-Union ($n=767$)	51.50	57.11 (+5.61)**	56.32 (+4.82)**	55.80 (+4.30)**	54.24 (+2.74)**	57.24 (+5.74)**	56.06 (+4.56)**	56.19 (+4.69)**
Spider-Union ($n=502$)	79.88	82.47 (+2.59)**	80.88 (+1.00)	81.47 (+1.59)	80.88 (+1.00)	81.87 (+1.99)	81.47 (+1.59)	80.88 (+1.00)

Table 4: [R1.W3] New in § 6.9. BEAVER per database and pooled, BASESQL with GPT-4.1 mini, top- $k=10$; the paper prints the pooled row. ** $p < 0.05$.

Arm	dw ($n=500$)	neutron ($n=509$)	nova ($n=527$)	pooled ($n=1,536$)
Baseline	7.8	33.2	22.6	21.3
+A	7.4 (-0.4)	34.2 (+1.0)	23.3 (+0.8)	21.7 (+0.5)
+P	8.6 (+0.8)	33.0 (-0.2)	24.1 (+1.5)	22.0 (+0.7)
+R	8.8 (+1.0)	34.0 (+0.8)	21.6 (-0.9)	21.5 (+0.3)
+A+P	9.0 (+1.2)	34.0 (+0.8)	25.4 (+2.8)**	22.9 (+1.6)**
+A+R	10.6 (+2.8)**	34.0 (+0.8)	23.3 (+0.8)	22.7 (+1.4)*
+P+R	9.2 (+1.4)	35.0 (+1.8)	22.8 (+0.2)	22.4 (+1.1)
+A+P+R	10.6 (+2.8)**	34.6 (+1.4)	23.0 (+0.4)	22.8 (+1.5)*

MR item 5. For the detailed comments from R2 (which are not numbered) please in your cover letter respond to those comments as if those comments were numbered as D1, D2, etc.

Ans: The detailed comments from Reviewer #2 are numbered as D1 to D15 and are answered individually in our response to Reviewer #2.

RESPONSE TO REVIEWER #1

W1. The main contribution is overstated. ... +R is SNAILS-style renaming (and underperforms SNAILS’s reported 5–10%), +P recombines schema linking/pruning with scoped retrieval, and semantic views for NL2SQL already ship widely and commercially.

Ans: Thanks for the comment and for bringing the related work. Please refer to our response to the **Meta-Reviewer Summary**.

W2. “Semantics-preserving at the schema level” does not imply query-correctness-preserving once the model is nudged to use the views ... Example 3 itself shows pre-joining through bond/molecule changes a COUNT from 498 to 377 ... so any question that should not include the join can be silently wrong.

Ans: The reviewer is correct that using a view containing an unnecessary join can change the rows being aggregated and therefore produce an incorrect result. This is a real concern in BIRD-Union because INNER JOIN removes rows whose join key is NULL. Across the 104 declared foreign-key relationships, independently applying a one-hop inner join on each key causes 88 of the 104 joins to drop at least one row; over all these edge-level tests, 1.1M of 7.8M rows (14.1%) are removed. Example 3 in the paper illustrates exactly this failure mode: the unnecessary join through molecule removes rows with a NULL molecule_id, changing the count from 498 to 377. This is why our +A provides semantic views alongside the original base tables rather than replacing them.

We further check whether +A makes the model more likely to choose an incorrect table set. Restricting to questions for which the gold table set is available through schema linking and the offered views, GPT-4.1 mini produces a final query with the wrong table set on 91 questions under the base schema, compared with 80 under +A. Thus, although an inappropriate view can introduce the multiplicity problem identified by the reviewer, we do not observe that exposing +A systematically increases incorrect table-set selection.

W3. ... the reported numbers do not have error bars, repeated runs, or significance tests ... the baselines are dated. New agentic text-to-SQL solutions (such as AutoLink, DeepEye, etc.) are not evaluated ... Spider 2.0 or BEAVER would be more convincing.

D2. ... provide mean $\pm$ std (or CIs) over multiple runs with fixed decoding/temperature, plus significance tests. This way, a reader can judge whether +0.7–+2.0 is real.

Ans: Thanks for the suggestion. We have added significance tests, repeated runs, the agentic baseline, and the BEAVER benchmark. Please refer to our responses to the meta reviewer (**MR item #3** and **MR item #4**).

D1. *Do the gains stay the same when BIRD evidence and column descriptions are included? ... does the proposed method still provide any benefit if the evidence or column descriptions were provided?*

Ans: In most cases, yes. Including BIRD’s evidence and column descriptions would substantially increase the performance of BASESQL, while our proposed transformations further improve the performance and achieve similar performance gains on GPT-4.1 mini and Gemini 2.5-flash-lite compared to the no-evidence setting. In Table 5 below, we list the evaluation results. External knowledge and the transformation are complementary: a description explains a column and thus reduces ambiguity, but it does not fully address the challenges in identifying join paths and localizing the relevant schema context.

Table 5: [R1.D1] New in § 6.7. BIRD-Union with evidence and column descriptions restored, BASESQL, $n=767$. **: McNemar $p < 0.05$; *: $0.05 \leq p < 0.10$.

Backbone	Baseline	+R	+A+P	+A+P+R
GPT-4.1 mini	61.15	62.71 (+1.56)	62.84 (+1.69)	63.23 (+2.09)*
Gemini 2.5-flash-lite	58.80	59.32 (+0.52)	62.84 (+4.04)**	62.71 (+3.91)**
GPT-5.4 mini	65.06	62.45 (-2.61)**	64.54 (-0.52)	65.32 (+0.26)

Table 6: [R1.D2] New in § 6.8. Mean $\pm$ std of EX over three independent runs of BASESQL +A+P+R; run 1 is the published one.

Benchmark	Backbone	run 1	run 2	run 3	mean $\pm$ std
BIRD-Union	GPT-4.1 mini	52.28	53.06	52.15	52.50 $\pm$ 0.49
BIRD-Union	GPT-5.4 mini	53.85	53.59	55.15	54.20 $\pm$ 0.84
BIRD-Union	Gemini 2.5-fl	52.28	52.67	52.67	52.54 $\pm$ 0.23
Spider-Union	GPT-4.1 mini	81.27	80.90	81.50	81.22 $\pm$ 0.30
Spider-Union	GPT-5.4 mini	84.26	85.06	84.66	84.66 $\pm$ 0.40
Spider-Union	Gemini 2.5-fl	83.07	83.67	83.67	83.47 $\pm$ 0.35

D3. *Please report offline cost (number of LLM calls for view-DDL synthesis and execution-equivalence-checked history rewriting), storage/maintenance for the logical/materialized views, and added inference latency from retrieval and longer prompts.*

Ans: Thanks for the suggestion. We have added a cost analysis in Section 6.10 in the revised paper. We summarize and clarify the three components below.

Offline cost. Transforming BIRD-Union (+R+A+P, 75 tables and 767 historical queries) requires 107 LLM calls in total, consuming 0.67M input and 0.19M output tokens (see Table 7 below). This includes 93 calls for view-DDL synthesis, 6 for rewriting SQL using the generated views, 6 for history rewriting under the transformed namespace, and 2 for rename synthesis/retry. We validate rewritten historical queries by executing them against the original gold SQL before accepting them.

Storage and maintenance. We clarify that the views used by +A are logical rather than materialized. Therefore, our implementation introduces no storage for materialized tuples and no view-refresh cost when the underlying database changes. The offline phase stores only the logical view definitions and transformed historical examples. We generate these once for the workload used in our experiments. If the workload distribution itself changes substantially, the logical design may need to be regenerated; maintaining it under a dynamically evolving workload is an interesting topic but is outside the scope of this paper.

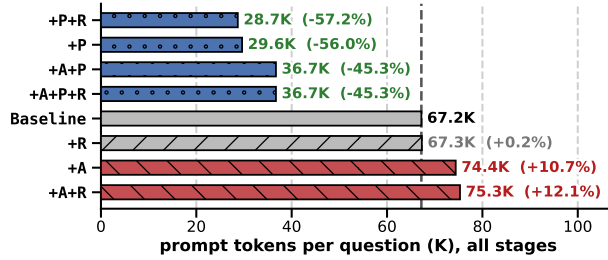
Online cost: We report prompt tokens per question in Figure 2, with a detailed breakdown across schema linking, SQL generation, and SQL revision in Table 8. Relative to the baseline, +R introduces negligible prompt overhead (+0.2%), +A increases prompt tokens by 10.7%, +P reduces them by 56.0%, and the full +A+P+R configuration uses 45.3% fewer tokens. We observe a similar reduction in prompt cost on BEAVER (supplementary material). We additionally measure inference latency for the baseline, +A, and the full +A+P+R configuration with GPT-4.1 mini on BIRD-Union. The baseline takes 7.10 s/question across the three LLM stages. Despite increasing prompt tokens by 10.7%, +A takes 7.12 s/question (+0.3%); the full configuration takes 7.09 s/question (−0.1%). Thus, in our API setting, the transformations introduce no measurable end-to-end LLM latency overhead, although their prompt-token costs differ substantially. Because the output length is around the same regardless of the setting, the majority of the latency is spent on the LLM producing the output.

Table 7: [R1.D3] New in § 6.10. Offline cost of one transformation of BIRD-Union (75 tables, 767 historical queries).

Phase	calls	input tok.	output tok.
View DDL synthesis	93	185,029	61,594
View-SQL rewrite	6	221,654	57,115
History rewrite	6	126,118	56,756
Rename synthesis (initial)	1	76,715	9,614
Rename synthesis (retry)	1	57,743	1,058
Total	107	667,259	186,137

Table 8: [R1.D3] New in § 6.10. Prompt tokens and measured latency per question on BIRD-Union with GPT-4.1 mini. Latency is measured for the baseline (BASESQL), +A: the individual operator with the largest prompt increase, and the full +A+P+R configuration.

Config	stage 1	stage 2	stage 3	total	Δ vs base
Baseline	23,315	21,898	21,982	67.2K	—
latency	2.47 s	2.20 s	2.43 s	7.10 s	—
+R	22,284	22,467	22,561	67.3K	(+0.2%)
+A	23,315	25,495	25,581	74.4K	(+10.7%)
latency	2.35 s	2.23 s	2.54 s	7.12 s	(+0.3%)
+P	23,315	3,073	3,162	29.6K	(-56.0%)
+R+A	22,284	26,466	26,565	75.3K	(+12.1%)
+R+P	22,284	3,186	3,279	28.7K	(-57.2%)
+A+P	23,315	6,670	6,754	36.7K	(-45.3%)
+R+A+P	22,284	7,186	7,278	36.7K	(-45.3%)
latency	2.31 s	2.39 s	2.39 s	7.09 s	(-0.1%)

**Figure 2: Prompt tokens per question, all stages (BASESQL, GPT-4.1 mini, BIRD-Union).**

D4. The history-aware results use a 50/50 i.i.d. split ... How sensitive are +P/+A to distribution shift (new question types/templates)? A temporal or template-disjoint split would test whether workload-aware partitioning generalizes beyond memorized access patterns.

Ans: Thank you for raising this point. We have added a robustness evaluation in Section 6.7 in the revised paper to test whether the workload-aware operators generalize beyond access patterns already observed in the historical workload. We keep the historical workload fixed and construct more challenging evaluation subsets by selecting a test question only when *no historical query uses the same set of tables with the same SQL shape*. We consider two definitions of query shape. **Skeleton** removes literals and table/column identifiers from the SQL query, leaving 599 evaluation questions; **keyword** keeps only the SQL-keyword structure, leaving 497 questions. Thus, the remaining questions differ from the historical workload in their combination of accessed tables and query structure.

These subsets are more difficult: baseline EX decreases by up to 5.5 percentage points. Nevertheless, +P and +A continue to improve performance across the three evaluated backbones (see Table 9 below). This suggests that the gains are not limited to memorizing exact historical access patterns and that the workload-derived logical design generalizes to previously unseen table-set/query-shape combinations.

Table 9: [R1.D4] New in § 6.7. Deltas over the baseline on the full test set and on the two template-disjoint subsets, BIRD-Union.
****:** McNemar $p < 0.05$; *****: $0.05 \leq p < 0.10$.

Backbone	Set	Baseline	+A	+P	+A+P	+R	+A+P+R
GPT-4.1 mini	Full (767)	49.80	(+1.04)	(+0.65)	(+2.61)**	(+0.65)	(+2.48)*
GPT-4.1 mini	Skeleton (599)	45.24	(+1.17)	(+1.00)	(+2.17)	(+2.17)	(+3.51)**
GPT-4.1 mini	Keyword (497)	44.27	(+0.60)	(+0.80)	(+2.01)	(+2.21)	(+2.01)
GPT-5.4 mini	Full (767)	52.80	(+2.74)**	(+0.26)	(+1.69)	(+0.13)	(+1.04)
GPT-5.4 mini	Skeleton (599)	47.58	(+2.17)	(+0.50)	(+1.34)	(+1.00)	(+1.50)
GPT-5.4 mini	Keyword (497)	46.88	(+2.82)**	(+0.80)	(+1.61)	(+0.80)	(+0.80)
Gemini 2.5-fl	Full (767)	48.11	(-0.13)	(+2.61)**	(+3.52)**	(+0.52)	(+4.17)**
Gemini 2.5-fl	Skeleton (599)	43.74	(-1.17)	(+2.00)	(+2.50)	(+1.00)	(+4.67)**
Gemini 2.5-fl	Keyword (497)	43.06	(-0.80)	(+2.21)	(+3.42)**	(+1.61)	(+4.63)**

D5. ...It would be more convincing if the authors can compare the base method (e.g., MAC-SQL) + A + P + R against one of the agentic solutions (e.g., AutoLink).

Ans: Thanks for the suggestion! We have added the experiment on AutoLink under every configuration on both BIRD-Union and Spider-Union. Please refer to our response to the Meta Reviewer (**MR item #4**).

RESPONSE TO REVIEWER #2

W1. The paper should discuss how the design approach differs from data marts in a data warehouse setting, where schema is denormalized as a star or snowflake schema ... data marts are not only designed for optimizing query performance but also for improving the usability for end-users.

Ans: We agree that data marts are closely related in motivation. A conventional star schema organizes data around a fact table at a chosen grain with associated dimension tables, while a snowflake schema further normalizes some dimensions [21]. Our approach shares the goal of improving the logical interface presented to the query author, but differs in both the target consumer and how the design is derived. A data mart is typically designed manually around business concepts, measures, dimensions, and grain; our logical layer is generated automatically from historical Text-to-SQL workloads and optimized for the representation exposed to an LLM, while keeping the original schema available.

FK-derived star-view baseline. To empirically compare against a schema design based only on dimensional join structure, we construct a star-schema inspired view set from the foreign-key graph: for each eligible relation, we create views joining it with its FK-connected tables. We emphasize that this is an automated FK-derived baseline rather than a manually designed data mart. It produces 46 views, compared with 64 workload-mined views under +A, and can re-express more gold queries (79.5% vs. 67.1%). However, it performs worse than +A on all three backbones and below the original schema on all three (see Table 10 below). The difference is that the FK graph captures what tables *can* be joined, whereas the historical workload provides evidence about what tables are actually useful to expose together to the LLM. Consistent with this, the FK-derived views contain more irrelevant tables per question (0.86 vs. 0.34) and exactly match the gold table set less often (40.3% vs. 58.8%).

Full denormalization. We also test the opposite extreme by fusing each database into a single relation. Because the result depends strongly on the choice of root table, we try every table as the root, construct the corresponding LEFT JOIN-based universal relation, and choose the root that maximizes gold-query rewriteability. Even under this favorable choice, three of the 11 databases yield no usable fused relation, and only 256 of 518 gold queries on the remaining databases can be rewritten against it. On those questions, EX falls from 48.8% to 37.9%. The main problem is the fixed grain imposed by the chosen root: one-to-many joins duplicate root-side values, while unmatched joins introduce NULL-padded columns. As a result, queries whose natural grain differs from that of the fused relation can return incorrect aggregates or selections. These comparisons suggest that simply denormalizing the schema cannot achieve our goal. Instead, a small set of workload-derived logical abstractions that simplify recurring access patterns while preserving the original relations for questions would help.

Table 10: [R2.W1/D3] New in § 6.5. FK-derived star view set vs. workload-mined views on BIRD-Union, structure and EX on three backbones.

	Base	Star (FK)	Mined (+A)
<i>Catalogue properties (schema-derived, backbone-independent)</i>			
views	—	46	64
tables per view	—	2.7	2.3
table set covers gold	—	88.9%	87.1%
table set equals gold	—	40.3%	58.8%
irrelevant tables offered	—	0.86	0.34
gold re-expressible on views	—	79.5%	67.1%
<i>Execution accuracy</i>			
GPT-4.1 mini	49.80	48.63 (-1.17)	50.85 (+1.04)
Gemini 2.5-flash-lite	48.11	43.29 (-4.82)	47.98 (-0.13)
GPT-5.4 mini	52.80	51.89 (-0.91)	55.54 (+2.74)

W2. The paper should also compare the results against the approaches proposed in [10] and [29].

Ans: We added a direct comparison with **EvoSchema** [44]. We construct BIRD-Union variants using *merge-table* and *merge-column* transformations, which are two relevant schema-evolution operators from EvoSchema, and evaluate them under the same Text-to-SQL pipeline. Merge-column is essentially neutral at 0.1% gain (49.9% vs 49.8%), while merge-table decreases EX by 4.4% (45.4% vs 49.8%). Interestingly, the merged schema improves table selection, but this does not translate into higher end-to-end accuracy: more than half of the newly introduced errors are projection errors. We include the merge-table result in Figure 8 as an additional prior-method comparison.

FootballDB [15] studies a different setting: real user questions over a single database with manually designed views. Our BIRD-Union and Spider-Union benchmarks instead merge multiple schemas and automatically derive the logical transformations from historical workloads. Because reproducing FootballDB’s approach would require manually constructing task-specific views for our benchmark schemas, there is no directly executable counterpart under our current experimental setup. We therefore discuss FootballDB as closely related evidence that logical schema design affects Text-to-SQL performance in Section 2, rather than presenting it as an incomparable numerical baseline.

W3. There is no human-validated evaluation about the correctness of the automatically generated schemas / views ... What is the upper bound for generation of S_R and S_A ? Given that most queries in BIRD cover on average 2.4 tables, one could simply generate views for 3-way-join queries of the most commonly used queries. I suspect that many queries would be improved with this simple baseline.

Ans: We agree that the generated logical schema should be validated independently of downstream Text-to-SQL accuracy. We therefore added a manual validation of the generated renames and additional validation/ablation of the generated views.

Human Validation of S_R . We manually inspected the generated identifier renames and found three semantically incorrect column renames (RF, RA, and Description), all from the thrombosis_prediction database. Only 5 of the 767 evaluation questions reference any of these columns.

Human-corrected S_R . We corrected the three names and reran all renamed configurations on the 82 thrombosis_prediction questions across the three backbones. The corrected mapping also provides an empirical estimate of the effect of eliminating the semantic rename errors we identified. The results are presented in Table 11 below. For the standalone +R configuration, correcting the three names changes EX from 26.83 to 24.39 on GPT-4.1 mini, from 30.49 to 25.61 on GPT-5.4 mini, and from 20.73 to 26.83 on Gemini 2.5-flash-lite. Thus, correcting the identified rename errors does not systematically improve +R, although the effect is model-dependent.

Validation and upper bound for S_A . For schema abstraction, each view is defined by its table set, join predicates/path, and join type; generated rewrites are kept only when they execute to the same result as the original historical query on the benchmark instance. We also evaluate an oracle-view condition in which the model receives the exact matching mined view. As shown in Table 12 below, this improves EX by +3.13, +5.09, and +1.30 points across the three backbones, compared with +1.04, +2.74, and -0.13 for normal +A. This shows the room for improvement in selecting the appropriate abstraction from the generated view set.

Three-way-join baseline. We also tested the reviewer’s suggestion that simply focusing on larger joins may capture most of the benefit. Restricting to views containing at least three tables leaves a matching view for only 45 of 767 questions and shows rough no performance gains across the three backbones (Table 12). The benefit of +A therefore does not come simply from exposing common three-table joins; the workload-mined views also need the smaller recurring abstractions that cover most questions.

Table 11: [R2.W3, R2.D10] New in §6.6. Effect of correcting the three wrong renames on every renamed configuration, thrombosis_prediction in BIRD-Union ($n=82$).

Backbone	Config	Baseline	Ours	Corrected	Corrected vs. Ours	Corrected vs. Baseline
GPT-4.1 mini	+R	23.17	26.83	24.39	(-2.44)	(+1.22)
GPT-4.1 mini	+A+R	23.17	23.17	23.17	(0.00)	(0.00)
GPT-4.1 mini	+P+R	23.17	21.95	21.95	(0.00)	(-1.22)
GPT-4.1 mini	+A+P+R	23.17	23.17	25.61	(+2.44)	(+2.44)
GPT-5.4 mini	+R	29.27	30.49	25.61	(-4.88)	(-3.66)
GPT-5.4 mini	+A+R	29.27	24.39	25.61	(+1.22)	(-3.66)
GPT-5.4 mini	+P+R	29.27	26.83	28.05	(+1.22)	(-1.22)
GPT-5.4 mini	+A+P+R	29.27	26.83	28.05	(+1.22)	(-1.22)
Gemini 2.5-flash-lite	+R	18.29	20.73	26.83	(+6.10)	(+8.54)
Gemini 2.5-flash-lite	+A+R	18.29	26.83	26.83	(0.00)	(+8.54)
Gemini 2.5-flash-lite	+P+R	18.29	23.17	20.73	(-2.44)	(+2.44)
Gemini 2.5-flash-lite	+A+P+R	18.29	21.95	23.17	(+1.22)	(+4.88)

Table 12: [R2.W3, R5.W2/D4] New in §6.5. View-set ablations on BIRD-Union, all mined views, three-or-more-table views only, and the oracle single view.

View pool	GPT-4.1 mini	GPT-5.4 mini	Gemini 2.5-fl
No view (baseline EX)	49.80	52.80	48.11
+A, all 64 mined views	+1.04	+2.74	-0.13
+A, ≥ 3 -table views only	+0.13	+1.04	+0.13
+A, oracle single view	+3.13	+5.09	+1.30
<i>Measurement subset, $n=459$ (absolute EX)</i>			
No view	48.80	51.20	47.49
+A, all 64 mined views	49.67	54.47	46.41
+A, oracle single view	54.03	59.69	49.67

W4. The link to the code is not public: <https://drive.google.com/drive/folders/...>

Ans: We apologize. The correct address is <https://anonymous.4open.science/r/Logical-Database-Design-CC0C>. The repository now includes the transformation code and the generated schema for reproducibility.

D1. Figure 1 should show the complete entity-relationship diagram with primary key/foreign key relationships.

Ans: Figure 2 (old Figure 1) is redrawn with the full primary key/foreign key relationships. The underline means primary key and the arrow means foreign key to referenced key. Primary and foreign keys are preserved in $\mathcal{S}'$ but not shown on the diagram here due to space.

D2. The running example with lexical ambiguity could be problematic since in the worst case all textual values would require renaming which is impractical. In previous work, the problem has been solved with inverted indexes over all base data tables with textual values.

Ans: +R renames schema identifiers, table and column names, not cell values. In the running example, label $\rightarrow$ carcinogenic_flag is an identifier rename, while the cell value ' + ' is unchanged. Therefore, the cost of +R scales with schema size rather than with the number of textual values in the database. Inverted indexes or other value-retrieval mechanisms are complementary: they can still be used to locate relevant cell values during Text-to-SQL generation, but they are orthogonal to the identifier-renaming problem studied by +R. We clarify this distinction in Section 3.3.

D3. The authors discuss the extreme cases of 3rd normal form and omni-view. However, they do not discuss the “middle ground” which is commonly used in data marts, namely star-schemas or snowflake schemas.

Ans: We agree and have added this middle ground explicitly. As described in our response to R2.W1 and in Section 6.5, we now discuss conventional star/snowflake data-mart design and evaluate an FK-derived star-view baseline against our workload-mined +A views, in addition

to the full-denormalization baseline. The comparison shows the distinction between schema-derived joinability and workload-derived access patterns.

D4. *Example 3 is a bit misleading. Good database design should enforce that there are no NULL keys in primary/foreign key relationships. With such a best practice approach, the demonstrated failure would not even occur.*

Ans: We have clarified this example. A primary key cannot be NULL, but a foreign-key column may legitimately be nullable; in that case, an unnecessary INNER JOIN removes rows whose foreign-key value is NULL. This is what happens in Example 3 and is why the count changes after joining through the additional table. More generally, the issue is not limited to NULL foreign keys. Even when every foreign key is non-NULL and satisfies referential integrity, adding an unnecessary one-to-many join can duplicate rows and therefore change aggregates such as COUNT or SUM. Thus, the example illustrates a broader point: a valid relationship does not imply that joining through it is semantics-preserving for every query. We have revised the discussion in Section 4.1 accordingly.

D5. *Example 4 is not clear. What exactly is the view atom_join_bond_join_molecule? Is it just a single table query as shown in Example 1?*

Ans: atom_join_bond_join_molecule is a logical view defined as a three-way INNER JOIN over atom, bond, and molecule using their declared foreign-key relationships, and it projects all columns from the three relations. We have made the view definition explicit in Example 1 and Example 4.

D6. *Moreover, the approach does not work for a potential subsequent question such as “What are the types of bonds and the names of the atoms (elements) of all connections of atom 19?” In this case you would need to join with the atom and bond tables.*

Ans: The reviewer is correct. A single abstraction is not intended to support every subsequent question. +A augments the original schema with workload-derived logical views; it does not replace or remove the base tables. Therefore, when a question requires a different combination of relations than the one represented by a particular view, the required base tables remain available, and the generated SQL can join them directly. This is also why we measure how often an offered view exactly matches the table set required by a question. An exact match indicates that the abstraction directly captures the question’s relational structure; when no such view exists, the model must rely on the base schema or combine the available relations differently. We clarify this behavior in Section 6.5.

D7. *Example 5: When asking for the elements of these atoms (and not just the atom_ID), the example in line 1 would break again. Moreover, the query in line 6 would even be correct.*

Ans: We thank the reviewer for pointing out this case. The partition shown in Example 5, {bond, connected}, is specific to the original question, which asks for the identifiers of the connected atoms. If the question instead asks for the elements of those atoms, then the atom table is also required, and the four-way query shown in the example would indeed be correct. We would like to clarify that our method does not reuse the same fixed partition for a subsequent question. Schema partitioning is performed per question: for each incoming question q , the system retrieves a question-specific sub-schema S_q^P . Thus, for the subsequent question (asking for the elements of these atoms), as the reviewer proposed, the intended behavior is to retrieve a different partition containing bond, connected, and atom, together with demonstrations associated with that partition. The reviewer’s example therefore highlights the possible risk of +P: if partition retrieval omits a table required by the new question, the pruned schema can no longer express the intended query. We quantify this risk on BIRD-Union: the retrieved partition contains all gold tables for 96.9% of the 767 test questions; we analyze the remaining 24/767 questions separately in Section 6.11.

D8. *... In a best practices design, primary/foreign key relationships are expressed explicitly. Hence, there is no need for LLMs to perform join-key reasoning since this information can be directly leveraged from the database schema.*

Ans: We agree that when primary/foreign-key relationships are declared, the model need not infer the join keys themselves; our implementation uses the declared keys directly. The purpose of +A is instead to narrow the remaining choice of which relationships to traverse for a question. The schema tells the model which tables can be joined, but not which subset of those joins corresponds to a recurring access pattern or is appropriate for the current question. +A uses the historical workload to expose such recurring join structures as logical views. This distinction, including cases where a table set admits multiple join paths, is discussed in more detail in Section 6.5.

D9. “When $\mathcal{H}$ is empty ... steps 2 and 4 instead run on demand at inference” — Step 2 can’t be run at inference time when $\mathcal{H}$ is empty. This is a contradiction to how you define step 2 which uses $\mathcal{H}$ as demonstrations for prompting.

Ans: The reviewer is correct. Our previous zero-shot description was inconsistent. In the old draft, we used the “few-shot demonstration from $\mathcal{H}$ ” in Section 5.2 for view DDL synthesis, which is not the same top- k demonstration used for SQL generation. We have revised that description in the paper for clarification. It is true that when $\mathcal{H}$ is empty, the abstraction procedure cannot run on historical examples in the same step, while this does not affect view DDL synthesis.

D10. “Prompt the LLM to rename each identifier into a Text-to-SQL-friendly alias” — How do you make sure these aliases are correct and meaningful when generated automatically?

Ans: We agree that automatically generated aliases should, ideally, be checked for semantic correctness. We now manually validate the generated renames; please see our response to your **R2.W3** and Section 6.6 in the revised paper. Among the generated aliases, we identify three semantically incorrect renames and additionally rerun the affected database after correcting them.

We also evaluate whether the generated names make the schema easier to align with user questions. Recent Text-to-SQL systems use textual embedding similarity between the question and schema elements for schema linking [14, 40]. We therefore compute the cosine similarity between the question and the relevant schema identifiers under the original schema, **+R**, and SNAILS. The average similarity increases from 0.690 under the original schema to 0.712 with **+R**, compared with 0.696 with SNAILS. This does not by itself establish semantic correctness, but provides complementary evidence that **+R** produces identifiers that are better aligned with the language used in the questions.

D11. “Validating $\mathcal{H}_R$ via execution equivalence ...” — Again, automatic validation is a good idea but how do you verify correctness?

Ans: Here, we use automatic validation specifically when rewriting the historical gold SQL into the namespace required by **+R**, **+A**, or **+A+R**. Since the original gold SQL is available, we execute both the original query on the base schema and the rewritten query on the transformed schema against the same database instance. A rewrite is accepted only if the two queries return identical results; otherwise, the system retries the rewrite up to a fixed maximum number of attempts. The same validation procedure is used for **+R**, **+A**, and **+A+R**. Thus, this check verifies that the generated historical rewrite preserves the gold query’s behavior on the benchmark database instance. We use this as an instance-based validation of the history rewrite, rather than as a formal proof of equivalence over all possible database instances.

D12. How were the 19 and 11 databases from Spider and BIRD chosen for Spider-UNION and BIRD-UNION, respectively?

Ans: BIRD-Union uses all 11 BIRD development databases. Spider-Union uses 19 out of the 20 Spider development databases with a total of 78 tables; we dropped the one that led to a table name collision. Please see Section 6.1.1 for the details.

D13. How was the 50:50 split of historical and evaluation questions done?

Ans: We randomly split each development set approximately 50:50 into a historical pool and a held-out evaluation set. The two sets are disjoint: no evaluation question is used when constructing any workload-derived transformation. This results in 767 historical / 767 evaluation questions for BIRD-Union and 502 / 502 for Spider-Union. We have clarified the splitting procedure and resulting sizes in Section 6.1 in the revised paper.

D14. Table 2: BaseSQL (50% Hist.) is clear. How much history do BaseSQL and the other systems include?

Ans: Unless otherwise stated, every pipeline and transformation configuration uses the same full historical pool: 767 questions for BIRD-Union, 502 for Spider-Union, and 1,534 for the newly added BEAVER. The corresponding held-out evaluation sets contain 767, 502, and 1,536 questions, respectively (the two-question difference for BEAVER results from splitting databases with an odd number of questions). The row labeled “BaseSQL (50% Hist.)” is a separate history-size ablation: it uses only 50% of the historical pool, i.e., 383 questions for BIRD-Union and 251 for Spider-Union. We have revised Section 6.1.5 to state these numbers explicitly.

D15. Figure 11 is not quite clear. Please explain better.

Ans: We have redrawn Figure 11 (now Figure 10 in the revised paper; also copied below) and rewritten its explanation in Section 6.11. The revised figure shows one bar per SQL component for each evaluated pipeline, rather than four bars per component. For each pipeline, we divide the evaluation questions into two sets: gained questions, which are incorrect under the baseline but become correct after applying **+A+P+R**, and lost questions, which are correct under the baseline but become incorrect after the transformation. For each SQL component,

the bar reports

component-match accuracy on gained questions – component-match accuracy on lost questions.

Thus, a positive value means the component is matched more often on questions the transformation fixes than on questions it breaks. For example, for BaseSQL/Gemini, the **Table set** bar is +27.3 points: table-set matching is 27.3 percentage points higher among the 74 gained questions than among the 42 lost questions. In contrast, the **Literals** difference is only +1.5 points. This makes the main pattern easier to see: gains are much more strongly associated with structural components such as table selection, join conditions, and selected columns than with lower-level components such as literals. We also add an example in Section 6.11 illustrating how to interpret one gained and one lost question.

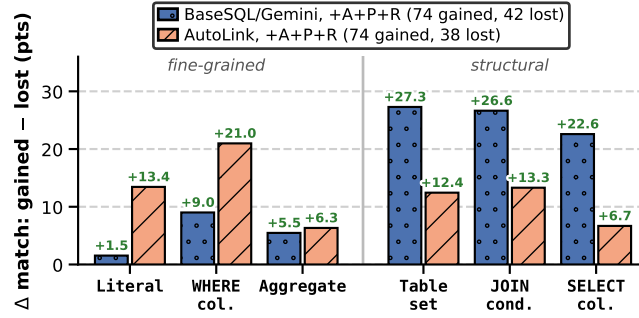


Figure 3: Where the losses come from: each pipeline’s component-match accuracy on the questions it gains minus on those it loses (BIRD-Union).

RESPONSE TO REVIEWER #5

W1 / D3. *The semantics of Schema Abstraction (+A) are underspecified ... it is unclear whether different join types, join predicates, preserved sides in outer joins, or cardinality-sensitive semantics are treated as distinct abstraction patterns ... A more precise formalization of view identity and stronger validation of generated views would improve the paper.*

Ans: Regarding the join semantics in +A: We now define a view by its table set, join predicates, and join type, following the treatment of join expressions in the materialized view-selection literature [20, 46]. Thus, two views over the same tables but with different join predicates or join types are treated as distinct abstraction patterns. On BIRD-Union, 13 of the 64 generated views involve table sets that appear with more than one join path in the historical workload; the less frequent paths account for only 1.2% of historical queries, and we instantiate the view using the majority path. In BIRD and Spider, all abstraction patterns observed in the historical workload use INNER JOIN, so outer-join preserved sides and NULL-padding semantics do not arise in our evaluated workloads.

Regarding validation of generated views: when declared foreign keys are available, we use them directly to construct or verify the join path; otherwise, we check the LLM-generated DDL to exclude Cartesian products. We further rewrite historical queries against the generated views and keep a rewrite only when it produces the same result as the original query on the benchmark database instance. We clarify that this is instance-dependent execution validation rather than a formal proof of semantic equivalence — same as the EX accuracy used in the Text-to-SQL metrics. See Sections 4.1 and 6.5 in the revised paper.

W2 / D4. *The framework may suffer from error propagation across transformations ... if an early transformation is incorrect, the error may affect later partition construction, view retrieval, demonstration selection, and final SQL generation. The paper would benefit from a more explicit analysis of how robust the pipeline is to upstream transformation errors.*

Ans: We agree that the composition may introduce the error propagation issue: if an upstream transformation removes information required by the query, later transformations may inherit or amplify that error. We therefore analyze such dependency in our pipeline, from +P to +A (added in Section 6.11 in the revised paper).

As reported in Table 13 below, +P removes at least one gold table for 24 of 767 questions (3.1%). We regard these cases as an upstream transformation error that could propagate into +A. However, we do not observe evidence that adding +A systematically amplifies the error: compared with +P, +A+P gains 29 questions and loses 14 on GPT-4.1 mini; even among the 24 questions where +P misses a gold table, +A+P answers 4 correctly compared with 3 for +P. The upstream error itself is nevertheless costly. Stratifying questions by intermediate state (Table 13) shows that +P achieves 51.7% EX accuracy when the partition contains all gold tables but only 12.5% when a gold table is missing; for +A+P, the corresponding accuracies are 53.6% and 16.7%. Thus, the main robustness requirement is high recall in the upstream schema-localization step: once a required table has been removed, downstream transformations generally cannot recover it.

Moreover, we analyze semantic errors introduced by +R separately in Section 6.6 in the revised paper; because +R is a bijective identifier mapping and does not remove tables or columns, its primary failure mode is incorrect semantic enrichment rather than loss of schema information that can cascade through subsequent transformations.

Table 13: [R5.W2/D4] New in § 6.11. EX breakdown by upstream error cases, BIRD-Union, GPT-4.1 mini ($n=767$).

Stratum	n	Baseline	+A	+P	+A+P
partition complete	743	50.6	51.5	51.7	53.6
partition <i>missing</i> a gold table	24	25.0	29.2	12.5	16.7

W3 / D5. *The novelty of +P and +R is limited. +P largely resembles prior schema linking and schema pruning techniques ... +R is also similar in spirit to prior work on schema naturalness, alias expansion, and metadata-based schema enrichment.*

Ans: We thank the reviewer for raising the concern. Please refer to our response to the **Meta-Reviewer Summary**.

The Case for Text-to-SQL Friendly Logical Database Design

Shi Heng Zhang
andy_zhang@sfu.ca
Simon Fraser University
Canada

Zhengjie Miao
zhengjie@sfu.ca
Simon Fraser University
Canada

Jiannan Wang
jnwang@tsinghua.edu.cn
Tsinghua University
China

ABSTRACT

Logical database design has traditionally optimized database schemas, including tables, columns, keys, constraints, and views, for correctness, integrity, and human-written application queries. LLM-based Text-to-SQL changes the consumer: the schema is now often read as text by a language model, so design choices that preserve database semantics can still change SQL-generation accuracy. We argue that this creates a new design objective alongside the classical ones — *LLM-friendly logical database design*, the property that a schema is easy for a language model to map from natural language to correct SQL — and treat it as the optimization target of this paper. We instantiate this objective by revisiting three established schema-design mechanisms: schema abstraction (+A: logical views that expose recurring join paths), schema partitioning (+P: workload-aware logical partitions that prune irrelevant context), and schema renaming (+R: descriptive identifiers that improve downstream column linking and predicate construction). The three operators compose, and each preserves the underlying database semantics. When historical question–SQL pairs are available, they guide both partitioning and abstraction; in zero-shot settings, renaming applies directly, and abstraction falls back to an ad-hoc per-question variant. We evaluate the resulting representations on BIRD-Union, Spider-Union, and BEAVER across five Text-to-SQL pipelines and three LLM backbones, with improvements of up to 5.7 execution-accuracy percentage points. The best transformation varies modestly across pipelines and models, with the full +A+P+R consistently improving; multiple operator combinations are competitive on each pipeline. These results show that LLM-friendly logical design is a practical and underexplored database-side optimization target, complementary to existing Text-to-SQL pipelines.

PVLDB Reference Format:

Shi Heng Zhang, Zhengjie Miao, and Jiannan Wang. The Case for Text-to-SQL Friendly Logical Database Design. PVLDB, 14(1): XXX-XXX, 2020.

doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://anonymous.4open.science/r/Logical-Database-Design-CC0C>.

1 INTRODUCTION

Logical database design depicts how real-world entities and relationships are organized into a relational database [17]. Traditionally,

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.

doi:XX.XX/XXX.XX

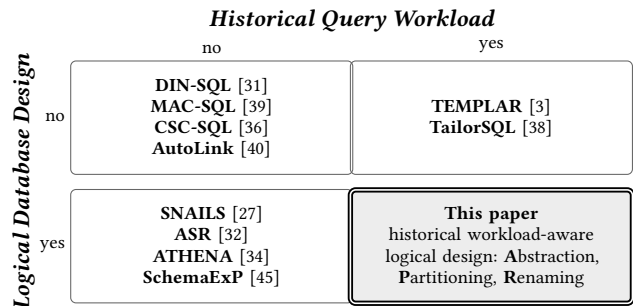


Figure 1: Positioning of prior work by whether it uses historical workloads and modifies the model-facing logical schema.

this process focuses heavily on system efficiency and optimization, prioritizing normal forms, data integrity, and eliminating redundancy over schema semantic clarity. While system efficiency is vital, the resulting schema is still the primary interface for consumption by developers. A well-designed schema should provide clear and domain-aligned abstractions, making it easier to reason about the underlying data and produce the correct SQL queries [15, 27].

In recent years, Text-to-SQL has emerged as a powerful tool to aid the query-writing process, allowing users to interact with databases using natural language questions. Driven by the rapid advancement of large language models (LLMs), this shift has fundamentally changed the consumption of the database schema: instead of being consumed exclusively by developers, the schema is now read and interpreted as textual context by the LLM. Modern Text-to-SQL systems [4, 14, 37, 41, 42] have substantially improved execution accuracy on challenging benchmarks like BIRD [26]. Despite this progress, most existing work treats the database schema as a fixed input, focusing exclusively on adapting the model, prompt, or decoding process to that schema. Prior works have shown that logical database design matters: SNAILS [27] renames identifiers for naturalness, semantic layers [32, 34] expose views, and query-logs to infer join paths [3]. EvoSchema [44] and FootballDB [15] empirically show that LLMs remain fragile to logical design choices, highlighting the urgent need for schema-level optimization.

These studies establish that schema representation matters for Text-to-SQL, but they leave open a different database-design question: *can the historical Text-to-SQL workload itself be used as a design signal to construct a better model-facing logical schema?* In this paper, we formulate this as *Text-to-SQL-friendly logical database design*: rather than treating the schema as a fixed input and adapting only the model or prompt, we optimize the logical representation exposed to the model using recurring access patterns in historical question–SQL pairs. The goal is not to introduce new schema operators individually, but to determine how established mechanisms—views, schema localization, and identifier enrichment—can be derived from the workload and composed into a persistent logical

layer that improves downstream Text-to-SQL accuracy. This shifts part of the optimization problem from the Text-to-SQL pipeline to the database representation that the pipeline consumes.

This observation motivates a complementary, *database-centric* perspective on Text-to-SQL. Rather than only asking how to improve the model for a given schema, we ask whether the schema itself can be transformed to better support the model. Specifically, given a database schema and a historical Text-to-SQL workload (optional), our goal is to construct a transformed schema that increases downstream Text-to-SQL accuracy.

To achieve this, we analyze the underlying process by which a language model reads and uses a database schema. To successfully generate a query, the model needs to understand three core dimensions of the schema: its relational structure (how tables connect), its relevance (which tables matter), and its semantics (what the tables and columns mean). To help the model navigate these three dimensions, we instantiate this design objective with three corresponding transformations: **Schema Abstraction** (+A) simplifies the relational structure by exposing recurring joined paths as logical views, **Schema Partitioning** (+P) organizes the schema into localized partitions to keep only the relevant schema and historical workload, and **Schema Renaming** (+R) reduces ambiguity by substituting table and column names with more descriptive identifiers. By applying these three operators, we adapt classic database optimization techniques to help language models achieve higher query execution accuracy.

EXAMPLE 1 (RUNNING EXAMPLE). Consider a molecular database, adapted from the BIRD benchmark [26], with relations such as `molecule`, `atom`, `bond`, and `connected`, as shown in Figure 2. Although this schema is reasonable from a normalized relational-design perspective, it can be difficult for a Text-to-SQL model to use directly.

To illustrate **Schema Abstraction** (+A), consider question Q_1 in Figure 2: “How many elements are there for single bond molecules?” The intended query requires the model to join `atom`, `bond`, and `molecule`. A baseline model may fail to produce this full path, generating an incomplete query over only `atom` and `bond`, e.g.,

```
atom ⋈ bond
```

thereby omitting the required context by the question. By applying +A, the framework instead exposes a semantic view of `atom` $\bowtie$ `molecule_id` `bond` $\bowtie$ `molecule_id` `molecule`,

```
atom_join_bond_join_molecule,
```

so that the model can express the query as a simple selection and aggregation over a single higher-level relation.

To demonstrate **Schema Partitioning** (+P), consider question Q_2 : “What atoms are connected in single type bonds?” The correct query only needs the relationship between `bond` and `connected`, projecting the connected atom identifiers:

```
1 SELECT connected.atom_id, connected.atom_id2
2 FROM bond JOIN connected ON ...
```

In contrast, the baseline model may over-join schema elements and introducing an incorrect four-way join with `atom` twice, while also not correctly projecting the connected atom identifiers. By applying +P, we restrict the context strictly to the relevant partition,

```
{bond, connected},
```

and retrieve demonstrations that reference this exact partition, guiding the model toward the correct join and projection patterns.

Finally, to show the impact of **Schema Renaming** (+R), consider question Q_3 , which states “How many chemical compounds are identified as carcinogenic?”. This information is in an ambiguous column `molecule.label`, making it difficult for the model to infer that this column is needed in a carcinogenicity predicate. By applying +R, we expose a clearer, semantically enriched identifier, e.g.,

```
molecule.label → Chem_Molecules.carcinogenic_flag,
```

consequently, establishing a direct lexical alignment between the natural-language question and the schema identifier.

The above example illustrates our core intuition: many Text-to-SQL failures are consequences of how the database schema is exposed to the model. This motivates *Text-to-SQL-friendly logical database design*: an additional design objective that asks how the model-facing schema should be organized so that an LLM can more reliably produce the correct SQL from the natural-language questions. This objective complements classical logical-design goals such as enforcing normal forms, ensuring data integrity, and application support, but optimizes for a new consumer of the schema: the language model. Ultimately, it seeks to make schema semantics clearer, reduce irrelevant context, and expose recurring access patterns as higher-level abstractions.

To show that this objective is actionable, we instantiate it as **Logical Database Design for Text-to-SQL**: a database-side framework that transforms the model-facing schema through three semantics-preserving operators: schema abstraction(+A), schema partitioning(+P), and schema renaming(+R). These operators revisit classical schema-design ideas under a new optimization target: improving Text-to-SQL generation accuracy. The framework supports both zero-shot and workload-aware settings and can be integrated with existing Text-to-SQL pipelines as a database-side enhancement, without changing model weights or model architecture.

Our main contributions are as follows:

- **Workload-driven logical database design for Text-to-SQL.** We formulate *Text-to-SQL-friendly logical database design* as an optimization problem over the model-facing schema representation. Our central perspective is to treat historical question-SQL workloads as a database-design signal: recurring access patterns are used to determine how the logical schema should be presented to downstream Text-to-SQL systems, rather than treating the schema as immutable.
- **A model-facing logical layer derived from the workload.** We instantiate this design space through three complementary transformations: schema abstraction (+A), which exposes recurring join paths as logical views; schema partitioning (+P), which localizes the schema and historical demonstrations to relevant regions; and schema renaming (+R), which exposes more descriptive identifiers. These mechanisms have antecedents in prior work; our contribution is to derive and compose them as a workload-driven logical layer while retaining the original database schema.
- **Comprehensive empirical evaluation across pipelines, models, and schemas.** We evaluate the design space on BIRD-Union, Spider-Union, and the enterprise benchmark BEAVER, across

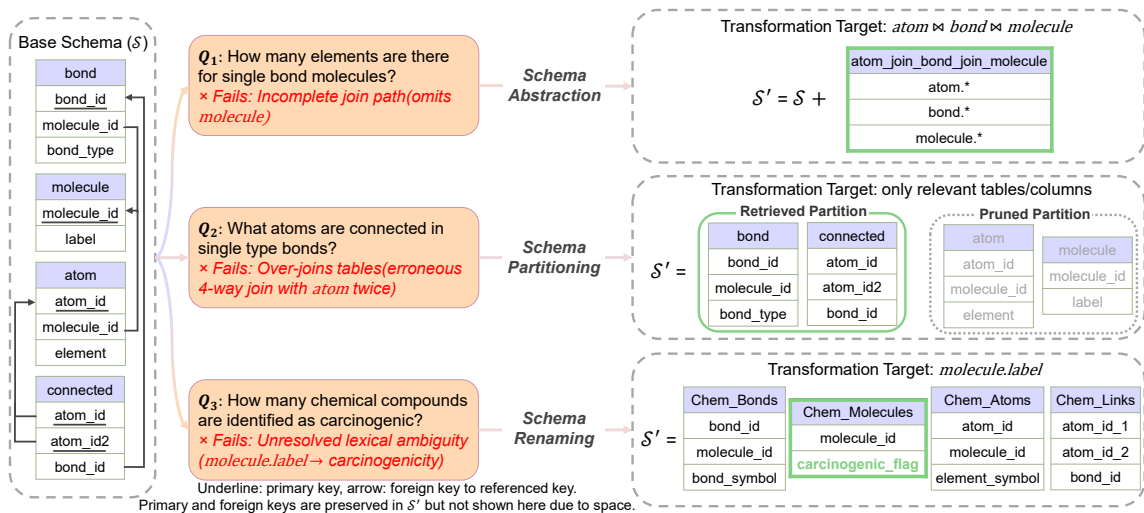


Figure 2: Running example illustrating how schema transformations mitigate complementary Text-to-SQL failure modes.

five Text-to-SQL pipelines and three LLM backbones. The transformed representations improve execution accuracy by up to 5.7 percentage points, including gains on the agentic AutoLink [40] pipeline. We compare the individual operators against their closest prior approaches and conventional schema alternatives, and analyze statistical reliability, workload shift, transformation errors, and offline/online cost to characterize when the transformations help and where they fail.

2 RELATED WORK

Traditional Text-to-SQL Approaches. While early encoder-decoder models established the task, today’s state-of-the-art relies on LLMs, sophisticated prompting, and self-repair. Prompt-centric approaches like DIN-SQL [31] and DAIL-SQL [16] improve performance through modular decomposition and execution-guided reasoning. To tackle enterprise-level databases, systems such as RSL-SQL [4], OpenSearch-SQL [41], CHESS [37], and RASL [14] explicitly ground the LLM using schema linking and retrieval augmentation. Concurrently, the field has advanced through complex reasoning architectures, including Monte Carlo Tree Search [24], test-driven refinement [42], and multi-agent pipelines leveraging corrective self-consistency [11, 23, 36, 39]. Despite these tremendous advancements, existing systems treat the database schema as a static input, which motivates our database-centric approach of dynamically optimizing the logical schema representation.

Importance of Schema for Text-to-SQL. A smaller but growing body of work recognizes that modifying the *schema* heavily alters downstream Text-to-SQL performance. A recent empirical study, SNAILS [27], demonstrates that semantically richer column and table identifiers can boost LLM accuracy by 5–10%. Similarly, EvoSchema [44] highlights that current LLMs are very susceptible to schema evolution and logical design shifts. [FootballDB \[15\] reaches the same conclusion from real user questions, showing that the same data under different logical models yields different accuracy.](#) Other efforts to bridge this semantic gap include schema expansion and pruning [45], attaching rich attribute annotations to DDL prompts [8], and generating synthetic foreign keys [25].

Schema expansion targets single tables whose cells pack several values into one string, and derives sub-field columns through a label and rules. ATHENA [34] suggests a hand-built domain ontology between the question and the schema but it cannot function properly without domain knowledge and human effort. While these works validate the importance of schema semantics, none of them suggest pre-computing joins as views to systematically apply dynamic logical transformations to augment the schema search space.

View Recommendation and Physical Design. View selection is a classic problem in physical database design. Foundational work [19] and systems like AutoAdmin [5] frame it as a cost-benefit optimization, trading maintenance overhead for query savings under storage constraints [6]. Later systems such as DynaMat [22], Microsoft’s automated design tools [1], and the DB2 Design Advisor [47] extended these ideas using workload-aware what-if costing and integer programming. While these traditional systems automate design to optimize *query execution*, we adopt this same philosophy to optimize *query generation* for the language model.

Workload-Aware Schema Adaptation. Building on the foundations of index and view selection, the database community has long utilized historical query workloads to drive structural schema optimizations. Examples include automatic vertical partitioning for OLTP workloads (e.g., AutoPart [2]) and graph-based data placement to minimize distributed joins (e.g., Schism [9]). Moreover, the vision of autonomous database systems [30] encourages monitoring query access patterns to dynamically restructure the underlying storage and logical layout. Traditionally, these systems utilize workload analysis exclusively to optimize query execution performance.

3 PROBLEM FORMULATION

3.1 Logical Database Design Meets Text-to-SQL

Logical database design creates a relational schema — tables, columns, data types, and integrity constraints — guided by entity-relationship modeling and normalization theories to reduce redundancy and preserve integrity. In Text-to-SQL, this schema is also part of the input that the generative model conditions on: the same design

choices that impact storage and integrity also dictate how easily a natural-language question can be expressed as SQL. Yet the effect of logical design on Text-to-SQL accuracy remains under-explored. For human developers, query writing becomes considerably easier when the query requires fewer joins under the given schema, uses self-descriptive identifiers, and demands little external knowledge; we hypothesize that the same factors translate to Text-to-SQL systems, which increasingly mimic human query-writing patterns.

3.2 Text-to-SQL-Friendly Database Design as an Optimization Problem

Database and Schema. Let $\mathcal{D}$ be a relational database with schema

$$\mathcal{S} = (\mathcal{R}, \mathcal{A}, \mathcal{C}),$$

where $\mathcal{R} = \{R_1, \dots, R_n\}$ is the set of relations, $\mathcal{A}$ is the set of attributes, and $\mathcal{C}$ represents the integrity constraints, including primary-key and foreign-key constraints.

The Task. Let $\mathcal{M}$ be a text-to-SQL system. Given a natural-language question q and a schema $\mathcal{S}$, the system generates a SQL query

$$y = \mathcal{M}(q, \mathcal{S}).$$

Let $\mathcal{W}$ denote a target workload of user questions. Optionally, let $\mathcal{H}$ denote historical workload associated with the same database, such as prior questions and query logs. In the zero-shot setting, $\mathcal{H}$ may be empty. We compute g from $(\mathcal{S}, \mathcal{H})$ at offline time and evaluate accuracy on a held-out workload $\mathcal{W}$ disjoint from $\mathcal{H}$. In the few-shot regime, $\mathcal{M}$ additionally receives top- k demonstrations $\mathcal{D}_q \subseteq \mathcal{H}$ selected by a retrieval function at inference time.

Our goal is to construct a transformed schema

$$\mathcal{S}' = g(\mathcal{S}, \mathcal{H}),$$

where g is a schema transformation function. As illustrated in Figure 3, we conduct this function g through a two-phase framework. In the Offline Schema Transformation phase, the base schema $\mathcal{S}$ and historical workload $\mathcal{H}$ are processed through a modular pool of transformations, namely Schema Abstraction (+A), Schema Partitioning (+P), and Schema Renaming (+R), to produce the optimized schema $\mathcal{S}'$. In the Online Prompt Assembly phase, this optimized schema acts as a plug-and-play enhancement that integrates into the flow of any arbitrary Text-to-SQL pipeline $\mathcal{M}$ to process the user question q and generate the query y .

Semantic preservation. Each of the three operator families preserves a different equivalence relation with the base schema. Schema Renaming g_R is a bijection on identifiers, equipped with a mapping dictionary $\mathcal{D}_R$ such that every query over the renamed schema is re-expressible over $\mathcal{S}$. Schema Abstraction g_A introduces logical views derivable from $\mathcal{S}$ via SQL composition, adding no new information. Schema Partitioning g_P produces a question-specific sub-schema $\mathcal{S}_q^P \subseteq \mathcal{S}$, restricting but never extending the relations exposed to the model. Therefore, these preservation modes guarantee that $\mathcal{S}' = g(\mathcal{S}, \mathcal{H})$ never alters the underlying database semantics; it only changes the logical representation presented to the model [15, 17, 18]. Formal definitions follow in Section 4.

EXAMPLE 2. Consider the base schema in Figure 2, with relations *molecule*, *atom*, *bond*, and *connected*. For question Q_1 "How

Table 1: Text-to-SQL-friendly logical design principles.

Classical design issue	Text-to-SQL bottleneck	Model-facing operator
Normalization preserves integrity but fragments data across relations	Complex join-path reasoning and bridge-table inference	Schema abstraction (+A)
Weak identifiers are compact but obscure data semantics	Lexical mismatches between the question and schema	Schema renaming (+R)
Full schemas ensure completeness but enlarge the search space	Irrelevant context distracts schema linking and retrieval	Schema partitioning (+P)

many chemical compounds are identified as carcinogenic?", the original input to the Text-to-SQL model is the base schema $\mathcal{S}$. Applying renaming g_R yields a schema with descriptive identifiers, e.g., *molecule.label* $\rightarrow$ *Chem_Molecules.carcinogenic_flag*. Applying partitioning g_P for Q_1 then restricts the model-facing schema to $\mathcal{S}_{Q_1}^P = \{\text{Chem_Molecules}\}$. The composition $\mathcal{S}' = (g_P \circ g_R)(\mathcal{S}, \mathcal{H})$ preserves the underlying database semantics by the modes above; it only changes the logical representation presented to the model.

Task Objective. Given $\mathcal{M}$, $\mathcal{W}$, we seek a transformation function g such that the resulting schema $\mathcal{S}' = g^*(\mathcal{S}, \mathcal{H})$ maximizes downstream Text-to-SQL accuracy:

$$g^* = \arg \max_g \text{Acc}(\mathcal{M}, \mathcal{W}, g(\mathcal{S}, \mathcal{H})), \quad (1)$$

where $\text{Acc}(\mathcal{M}, \mathcal{W}, \mathcal{S}')$ measures the quality of SQL queries generated by $\mathcal{M}$ over workload $\mathcal{W}$ when conditioned on schema $\mathcal{S}' = g(\mathcal{S}, \mathcal{H})$, and can be instantiated as

$$\text{Acc}(\mathcal{M}, \mathcal{W}, \mathcal{S}') = \frac{1}{|\mathcal{W}|} \sum_{q \in \mathcal{W}} \text{Score}(\mathcal{M}(q, \mathcal{S}'), y_q^*), \quad (2)$$

where y_q^* is the reference SQL for question q , and Score is an evaluation function such as execution accuracy or semantic equivalence.

Throughout this paper, g is constructed by combining the three operator families introduced above: each configuration applies a subset of $\{g_A, g_P, g_R\}$ jointly, and the seven non-empty combinations are evaluated as distinct configurations in our experiments. This formulation treats logical schema design itself as an optimization dimension for Text-to-SQL. Rather than only adapting the Text-to-SQL model to the schema, we seek to optimize the schema representation presented to the model.

3.3 Design Goals and Constraints

A transformed schema for Text-to-SQL should satisfy three goals: preserve the semantics of the original database, improve the schema’s accessibility for natural-language-driven query generation, and remain compatible with existing Text-to-SQL pipelines. These goals, however, are not fully aligned. Improving one aspect of the schema representation may introduce costs in another, making schema design for Text-to-SQL an inherently constrained optimization problem. As summarized in Table 1, our framework navigates these constraints by mapping classical design issues to their corresponding Text-to-SQL bottlenecks, and resolving them through three targeted model-facing operators.

Normalization vs. reasoning burden. Logical schemas used in practice are often normalized, e.g., to Third Normal Form (3NF) or Boyce–Codd Normal Form (BCNF), in order to reduce redundancy and preserve data integrity [17]. While such designs are desirable from a classical database perspective, they often increase the reasoning burden for Text-to-SQL systems. Even relatively simple user

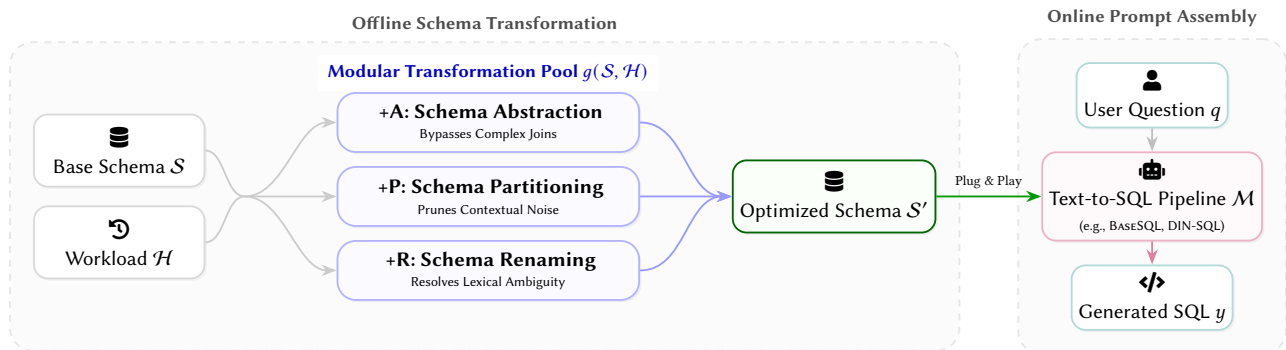


Figure 3: The proposed database-centric Text-to-SQL framework. The transformation function g utilizes offline schema transformation and online prompt assembly to construct an optimized representation S' for any downstream pipeline M .

questions may require multi-table joins, bridge tables for many-to-many relationships, or nontrivial foreign-key traversal before the intended query structure becomes apparent.

In the running molecular example, the question about single-bond molecules requires the model to recover an atom-bond-molecule access path before it can perform the final aggregation. The challenge is therefore not to abandon normalization, but to expose higher-level logical abstractions that reduce unnecessary join-path reasoning while preserving query semantics.

Semantic clarity vs. lexical ambiguity. A second constraint arises from the naming and organization of schema elements. In many real databases, table and column identifiers are abbreviated, overloaded, or only weakly aligned with natural-language concepts. Such lexical ambiguity creates friction for schema linking, since the model must infer the intended meaning of schema elements indirectly. For instance, an identifier such as `label` in the running example provides little direct evidence that it encodes carcinogenicity. **Renaming(+R) changes table and column names only, never cell values.** More descriptive names, enriched metadata, or semantically clearer attribute organization can substantially improve alignment between user questions and schema structure. However, such enrichment must be applied carefully: overly aggressive rewriting may distort established schema conventions or introduce inconsistencies across related schema elements.

Schema completeness vs. contextual noise. A third trade-off concerns schema scope. Exposing the full database schema ensures that all potentially relevant information is available to the model, but it also enlarges the search space and introduces substantial irrelevant context. This problem becomes especially pronounced in real-world, large, enterprise-level schemas, where most questions touch only a small subset of relations. In the running example, the question about connected atoms can be answered from a small schema neighborhood involving `bond` and `connected`, but exposing additional relations may encourage unnecessary joins or incorrect projections. At the same time, excessive pruning risks omitting information required to answer the query [15]. A Text-to-SQL-friendly logical design must therefore balance completeness against locality, exposing enough context to support correct query generation while suppressing irrelevant schema regions.

Implications for schema transformation. These trade-offs motivate the three operators detailed in Section 4: schema abstraction shortens rigid join paths; schema partitioning isolates coherent

schema neighborhoods; schema renaming improves identifier clarity. Together, they shift part of the reasoning burden from the model’s inference process into the logical schema representation itself, constrained by the semantic-preservation modes above.

4 SCHEMA TRANSFORMATION

We consider a transformation space that improves Text-to-SQL performance by modifying the logical schema representation exposed to the model while preserving the semantics of the underlying database. In our framework, transformations fall into three broad categories: schema abstraction, schema partitioning with workload-aware organization, and metadata enrichment via schema renaming. These operators may be applied independently or in combination to construct a Text-to-SQL-friendly schema representation.

To illustrate our framework throughout this section, we employ a **running example** based on a real-world chemistry database. The base schema (S) consists of four highly-normalized tables with generic identifiers:

- `molecule` (`molecule_id`, `label`)
- `atom` (`atom_id`, `molecule_id`, `element`)
- `bond` (`bond_id`, `molecule_id`, `bond_type`)
- `connected` (`atom_id`, `atom_id2`, `bond_id`)

Figure 2 visually summarizes how our three transformation techniques operate on this base schema to produce a Text-to-SQL-friendly variant (S').

4.1 Schema Abstraction

A central challenge in Text-to-SQL is the gap between low-level relational structure and high-level user intent. Normalized schemas (typically in 3NF or BCNF [17]), such as those in SPIDER [43] and BIRD [26] benchmarks, distribute the information needed for a single question across multiple relations, forcing the model to infer the join path before selection, aggregation, or projection in the query. One natural way to reduce this reasoning burden is to expose higher-level logical abstractions through views. In extreme cases, one could imagine an “omni-view” that joins every relation and contains all attributes across the schema, thereby eliminating the need for explicit JOIN reasoning. In practice, however, such a representation is neither semantically clean nor operationally reliable; the example below illustrates why it is not the answer.

EXAMPLE 3. The ground-truth query Q_{corr} returns 498 connections for atom 19. Adding a join through the bonds and molecule tables, as in Q_{wrong} , deflates the count to 377. The cause is that INNER JOIN

silently drops every tuple whose join key is NULL on either side—so rows in `connected` whose `bond_id` is NULL (or whose corresponding `bonds.molecule_id` is NULL) disappear from the result, even though the selection predicate has not changed.

```

1 102 How many connections does the atom 19 have?
2  $Q_{corr} = \text{SELECT COUNT}(T.bond\_id) \text{ FROM } connected \text{ AS } T$ 
3  $\text{WHERE SUBSTR}(T.atom\_id, -2) = '19';$ 
4
5  $Q_{wrong} = \text{SELECT COUNT}(T.bond\_id) \text{ FROM } connected \text{ AS } T$ 
6  $\text{JOIN bond as b ON b.bond\_id} = T.bond\_id$ 
7  $\text{JOIN molecule as m on m.molecule\_id} = b.molecule\_id$ 
8  $\text{WHERE SUBSTR}(T.atom\_id, -2) = '19';$ 
9
10  $Q_{corr}$  is 498,  $Q_{wrong}$  is 377 because of joining molecule.

```

Our approach, therefore, synthesizes a set of workload-relevant semantic views that simplify common join patterns without collapsing the schema into a single denormalized table. The objective is to expose higher-level logical access paths that are easier for the model to align with natural-language questions, while retaining sufficient structure to preserve correct query semantics compared to the “omni-view” solution.

Formally, we model schema abstraction as an operator

$$g_A : (\mathcal{S}, \mathcal{H}) \rightarrow \mathcal{S}^A,$$

where $\mathcal{S}^A$ augments the schema with higher-level logical abstractions, such as derived semantic views, that simplify common access patterns while preserving the semantics of the underlying database.

EXAMPLE 4. The example below shows the failure that schema abstraction fixes on the single-bond-molecules question: the baseline generates the incorrect query Q_{wrong} that misses the molecule table, while the semantic view `atom_join_bond_join_molecule` (same view that was introduced in Example 1) pre-computes the join path the question requires, allowing the model to answer the question through a single logical relation.

```

1 259 How many elements are there for single bond molecules?
2  $Q_{corr} = \text{SELECT COUNT}(\text{DISTINCT element})$ 
3  $\text{FROM atom\_join\_bond\_join\_molecule}$ 
4  $\text{WHERE bond\_type} = '-';$ 
5
6  $Q_{wrong} = \text{SELECT COUNT}(\text{DISTINCT } T1.element) \text{ FROM atom AS } T1$ 
7  $\text{INNER JOIN bond AS } T2 \text{ ON } T1.molecule\_id = T2.molecule\_id$ 
8  $\text{WHERE } T2.bond\_type = '-';$ 

```

Constraint. Per the semantic-preservation typing established in Section 3, views in $\{V_1, \dots, V_n\}$ must be derivable from $\mathcal{S}$ via SQL composition, so g_A introduces no new information. The design challenge is to balance *specificity* (a narrow view captures fewer questions) and *coverage* (broader views drift toward the omni-view, risking semantic distortion through indiscriminate joins).

4.2 Schema Partitioning

Large schemas introduce another difficulty for Text-to-SQL systems: most questions touch only a small subset of the available relations, yet the model is often exposed to the full schema. This creates substantial noise for schema linking and enlarges the search space for query generation. For example, in the BIRD benchmark [26], databases contain an average of roughly 40 tables, whereas a typical question requires access to only a small fraction of them (2.4 on average). This mismatch suggests that much of the schema is irrelevant for any given query and may distract the model from the truly relevant relations [15].

To address this issue, we first *localize* the relevant partition of the schema for a given question, and then organize it into smaller, semantically coherent partitions. We refine this organization using table co-occurrence patterns and recurring join structures observed in past queries. Each partition may additionally be annotated with auxiliary information such as typical join paths, representative query templates, or salient projection patterns.

Given a question q , we define schema partitioning as the task of selecting a question-specific sub-schema

$$\mathcal{S}_q^P = L(q, \mathcal{S}, \mathcal{H}) \subseteq \mathcal{S},$$

where $\mathcal{H}$ denotes historical workload information. The goal is to retain the tables, columns, and join relationships most relevant to answering q while suppressing irrelevant schema partitions.

Beyond reducing schema-level noise during schema linking, the partitioning provides a natural retrieval unit for historical demonstrations: the system first localizes the schema partition for q , then retrieves demonstrations from queries assigned to that partition. The example below illustrates the benefit of schema partitioning.

EXAMPLE 5. On the single-type-bond question below, the baseline produces an unnecessary four-way join in Q_{wrong} . In contrast, partitioning the schema to

$$\mathcal{S}_q^P = \{\text{bond}, \text{connected}\}, \quad \text{Partition}_1 = [\text{bond}, \text{connected}],$$

and drawing demonstrations from that retrieved partition, `Partition1`, expose a simpler and more appropriate query pattern, recover the correct projection, leading to the correct query Q_{corr} .

```

1 210 What atoms are connected in single type bonds?
2  $Q_{corr} = \text{SELECT } T2.atom\_id, T2.atom\_id2 \text{ FROM bond AS } T1$ 
3  $\text{INNER JOIN connected AS } T2 \text{ ON } T1.bond\_id = T2.bond\_id$ 
4  $\text{WHERE } T1.bond\_type = '-';$ 
5
6  $Q_{wrong} = \text{SELECT } T3.element, T4.element \text{ FROM bond AS } T1$ 
7  $\text{INNER JOIN connected AS } T2 \text{ ON } T1.bond\_id = T2.bond\_id$ 
8  $\text{INNER JOIN atom AS } T3 \text{ ON } T2.atom\_id = T3.atom\_id$ 
9  $\text{INNER JOIN atom AS } T4 \text{ ON } T2.atom\_id2 = T4.atom\_id$ 
10  $\text{WHERE } T1.bond\_type = '-';$ 
11
12 History retrieved from the partitions:
13 111 Identify all connected atoms with a triple bond.
14  $\text{SELECT } T2.atom\_id, T2.atom\_id2 \text{ FROM bond AS } T1$ 
15  $\text{INNER JOIN connected AS } T2 \text{ ON } T1.bond\_id = T2.bond\_id$ 
16  $\text{WHERE } T1.bond\_type = '#';$ 

```

Constraint. Per Section 3, $\mathcal{S}_q^P$ is a strict subset of $\mathcal{S}$ (no new tables introduced), and the partition selection must preserve recall of the relations required to answer q . Restricting the demonstration pool to historical queries assigned to the retrieved partitions extends the same locality discipline to retrieval, so the few-shot context shares the partition of the model-facing schema.

4.3 Schema Renaming

The effectiveness of Text-to-SQL systems depends heavily on the semantic clarity of schema identifiers. In practice, table and column names often vary widely in interpretability, ranging from intuitive descriptors to cryptic abbreviations that require domain knowledge to decode. As observed by prior work such as SNAILS [27], identifier quality has a direct impact on schema understanding and downstream Text-to-SQL performance.

This dependence is particularly important for LLM-based systems, which rely heavily on lexical and semantic alignment between the natural-language question and the schema representation.

When identifiers are opaque, overloaded, or weakly descriptive, the model must infer their meaning from context, increasing the risk of schema-linking errors. The following example illustrates this issue.

EXAMPLE 6. *On the carcinogenic-compounds question below, the generic table name molecule and the ambiguous column identifier label = ‘-’ defeat the baseline (incorrect Q_{wrong}); renaming the identifiers to Chem_Molecules.carcinogenic_flag = ‘+’ enables the correct query Q_{corr} .*

```

1 291 How many chemical compounds in the database are
    ↪ identified as carcinogenic.
2  $Q_{corr}$  = SELECT COUNT(*) FROM Chem_Molecules
3 WHERE carcinogenic_flag = '+';
4
5  $Q_{wrong}$  = SELECT COUNT('molecule_id') FROM 'molecule'
6 WHERE 'label' = '-';

```

Formally, we model metadata enrichment and schema renaming as an operator

$$g_R : S \rightarrow S^R,$$

where S^R preserves the original schema semantics while exposing more descriptive identifiers and metadata.

Constraint. According to Section 3, g_R is bijective on identifiers, and the mapping dictionary $\mathcal{D}_R$ propagates foreign-key constraints onto S^R . These ensure that every query over S^R remains re-expressible over S , so the rename is a non-destructive logical layer rather than a structural rewrite of the underlying database.

5 SYSTEM IMPLEMENTATION

5.1 System Architecture Overview

We instantiate the operators g_A , g_P , and g_R defined in Section 4 as a single middleware layer between the physical database and the Text-to-SQL pipeline. Classic Text-to-SQL pipelines force language models to interact directly with physical database schemas, which are engineered to reduce redundancy rather than for Text-to-SQL friendliness. To bridge this semantic gap, our framework operates as a middleware layer that decouples the physical database schema (S) from a highly optimized logical representation (S^R).

The architecture is divided into two operational phases to shift the computational burden to offline preparation. During the offline phase, the framework analyzes S alongside optional historical query workloads ($\mathcal{H}$) to automatically generate S^R and S^A . Specifically, this phase generates the logical layer for Schema Renaming (+R) and Schema Abstraction (+A), identifies table partitions for Schema Partitioning (+P), and performs the necessary query rewriting for +R and +A. **All objects are logical views with no materialization, so the transformation adds no storage.**

During online inference, after obtaining the schema-linking result, the system dynamically retrieves the most relevant partitions and pre-joined views to construct a highly localized prompt context. This decoupled architecture assists the language model by removing the complexities of ambiguous physical identifiers and overwhelming search spaces, enabling highly accurate SQL generation without altering the physical schema.

5.2 Schema Abstraction Architecture (+A)

The +A implementation proceeds in four steps (illustrated by the dashed bounding box in Figure 4):

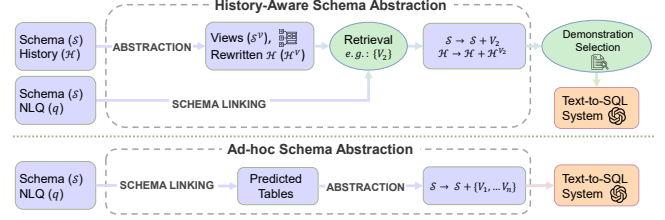


Figure 4: The Schema Abstraction (+A) Architecture.

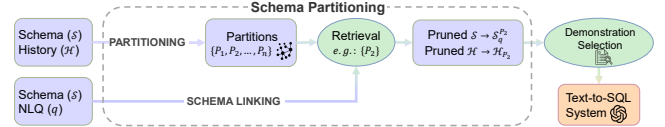


Figure 5: The Schema Partitioning (+P) Architecture.

- Mine candidate views.** Run frequent-itemset mining in the offline stage over the AST-extracted table sets in $\mathcal{H}$ to obtain recurring multi-table join patterns $\{V_1, \dots, V_n\}$.
- Synthesize view DDL.** For each candidate, we prompt the LLM to generate the view DDL, with a few fixed demonstrations that show the expected output format. If the LLM struggles to provide a parse-able DDL, and when foreign keys are explicit, we fall back on a deterministic code-based generator to enforce the correct execution of the join path. Then, we drop any candidate whose DDL would yield a Cartesian product and cache and persist the rest, forming $S^A = S \cup \{V_1, \dots, V_n\}$. **A view is identified by its table set, join type and join predicates. If the same table set is joined in more than one way in $\mathcal{H}$, each pattern yields its own view; when $\mathcal{H}$ is empty, the view defaults to an INNER JOIN.**
- Rewrite history.** If the abstracted schema was never used in the few-shot demonstrations, the model might not be able to identify its use. Therefore, we iteratively prompt the LLM to rewrite $\mathcal{H}$ against S^A and accept only rewrites that are execution-equivalent to the original, producing $\mathcal{H}^A$.
- Inject at inference.** For each incoming question, append every view in S^A that contains a selected table from schema linking (e.g., $\{V_2\}$ in the figure) to the LLM context, and merge the corresponding $\mathcal{H}^{V_i}$ (queries that access the view V_i) into the demonstration pool.

The LLM-driven step 2 has the distinct advantage of inferring undeclared semantic foreign keys (e.g., debit_card_specializing), while the code-based fallback enforces deterministic execution when foreign keys are declared. Cached ad-hoc views persist across queries: when a subsequent query requires the same set of tables, the system retrieves the cached view, transitioning ad-hoc synthesis into a persistent optimization strategy. By allowing the LLM to query the augmented S^A in step 4, this approach eliminates complex join-key reasoning and reduces the likelihood of join errors.

When $\mathcal{H}$ is empty (no historical workload; zero-shot in prompting), steps 1 and 3 are skipped; steps 2 and 4 instead run on demand at inference, taking the tables identified during schema linking as the input to step 2 (the bottom path of Figure 4).

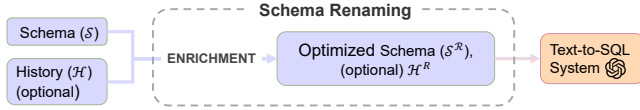


Figure 6: The Schema Renaming (+R) Architecture.

5.3 Schema Partitioning Architecture (+P)

The +P implementation proceeds in four steps (illustrated by the dashed bounding box in Figure 5):

- (1) **Extract table sets.** Parse the Abstract Syntax Tree (AST) of every query in $\mathcal{H}$ and record the referenced tables.
- (2) **Partitions offline.** Treat the extracted table sets as a frequent-itemset problem; form partitions $\{P_1, \dots, P_n\}$ from table combinations whose frequency exceeds a tunable threshold τ , and assign each query in $\mathcal{H}$, alongside Common Join Paths extracted from $\mathcal{H}$, to the partitions whose tables they reference.
- (3) **Retrieve at inference.** Pass the question q through a schema-linking module to predict the required tables; retrieve any partition containing at least one predicted table (e.g., $\{P_2\}$).
- (4) **Prune schema and history.** Restrict the model-facing schema to $S_q^P = \bigcup_{P_i \text{ retrieved}} P_i$ (e.g., $S_q^{P_2}$); restrict the demonstration pool to historical queries assigned to the retrieved partitions (e.g., $\mathcal{H}_{P_2}$); select top- k demonstrations by semantic similarity strictly within this bounded pool.

By treating offline partitioning as a frequent-itemset problem in step 2 rather than relying on static foreign-key topologies or probabilistic graph communities, the system ensures that the partitions represent genuine, frequently utilized access paths grounded in $\mathcal{H}$. The partition-retrieval rule in step 3 maximizes recall while keeping the model-facing schema focused. The hard boundary imposed on the demonstration pool in step 4 constrains the few-shot demonstrations to the exact same partitioned table space as S_q^P , providing a focused context with less noise. **When +P misses a gold table, adding +A on top does not propagate the error (Section 6.11).**

5.4 Schema Renaming Architecture (+R)

The +R implementation proceeds in five steps (Figure 6):

- (1) **Parse the physical schema.** Extract identifiers and foreign-key constraints from the DDL of S into a structured prompt.
- (2) **Generate descriptive aliases.** Prompt the LLM to rename each identifier into a Text-to-SQL-friendly alias; when $\mathcal{H}$ is available, include its query patterns in the prompt to disambiguate cryptic identifiers (e.g., mapping A2 to district_name).
- (3) **Instantiate the logical layer.** Instantiate S^R as 1:1 logical views over the base tables (verified by a lightweight integrity check), and construct the mapping dictionary $\mathcal{D}_R : (t_{phys}, c_{phys}) \rightarrow (t_{opt}, c_{opt})$ that propagates the base tables' foreign-key constraints onto S^R .
- (4) **Rewrite history.** Iteratively prompt the LLM to rewrite $\mathcal{H}$ against S^R , accepting only rewrites validated by execution equivalence with the original, producing $\mathcal{H}^R$.
- (5) **Inject at inference.** Provide S^R and $\mathcal{H}^R$ to the LLM, so it operates entirely within the optimized semantic space.

Schema Renaming is non-destructive: the physical schema is untouched and steps 1–4 only add a logical layer on top. Because S^R has a 1:1 mapping over S , the SQL generated at step 5 executes

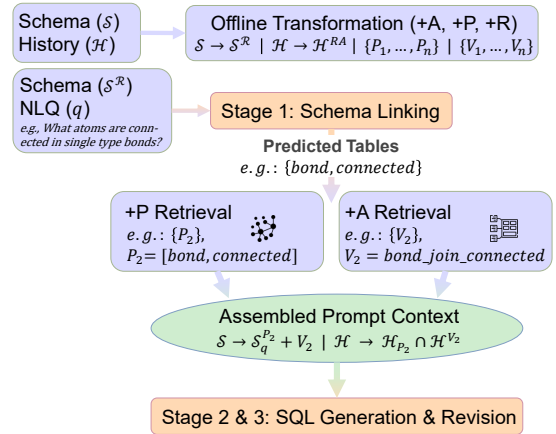


Figure 7: The online prompt assembly integrating +A+P+R.

directly on the underlying engine without any post-hoc parsing or rewriting. Validating $\mathcal{H}^R$ via execution equivalence in step 4 is critical: $\mathcal{H}^R$ is injected into the prompt at step 5 as few-shot demonstrations, and undetected semantic drift in the rewrites would propagate into the generated SQL. **Section 6.6 discusses about +R's validation, and Section 6.11 shows that +R errors do not propagate to +P or +A.**

5.5 Base Text-to-SQL Pipeline and Online Prompt Assembly

5.5.1 Base Text-to-SQL Pipeline (BASESQL). To evaluate and isolate the impact of logical schema optimization from complex prompt engineering heuristics, we evaluate our framework using a streamlined three-stage baseline pipeline, BASESQL. Stage 1 performs initial schema linking (inspired by DIN-SQL [31]). Stages 2 and 3 handle SQL generation and revision using CHES [37] prompts, leveraging Stage 2's query's runtime diagnostics for Stage 3 to correct structural flaws and produce the final SQL query.

5.5.2 Full Pipeline Prompt Assembly (+A+P+R). Our framework integrates seamlessly into the BASESQL baseline (or any Text-to-SQL pipeline) by dynamically assembling an optimized schema and prompt context along the standard flow. As illustrated in Figure 7, this integration occurs across three assembly points:

1. Offline Transformation (+A, +P, +R): Computed entirely offline before any query is processed, +R transforms the physical schema S into the optimized schema S^R and rewrites the historical workload $\mathcal{H}$ into $\mathcal{H}^R$. Utilizing S^R and $\mathcal{H}^R$, the offline phases of +P and +A generate the partitions $\{P_1, \dots, P_n\}$ (e.g., the partition with [bond, connected]), logical views $\{V_1, \dots, V_n\}$ (e.g., bond_join_connected), and rewritten history $\mathcal{H}^{RA}$.

2. Dynamic Schema Assembly (+A, +P): Between stage 1 and 2, the predicted tables trigger the retrieval modules. +P retrieves relevant partitions (e.g., P_2), pruning the schema down to S_q^P (e.g., $S_q^{P_2}$) and restricting the historical pool to $\mathcal{H}_{P_2}$ for top- k retrieval. Concurrently, +A appends relevant logical views (e.g., V_2) and their corresponding history (e.g., $\mathcal{H}^{V_2}$) to the historical pool.

3. Stage 2 & 3: SQL Generation and Revision: The final assembled context, comprising the unified schema ($S_q^{P_2} + V_2$) and the focused top- k history ($\mathcal{H}_{P_2} \cap \mathcal{H}^{V_2}$), is provided to the LLM. Both

generation and revision operate within this identical optimized semantic space, with Stage 3 additionally incorporating the runtime execution feedback of the generated query from stage 2.

6 EXPERIMENTS

We evaluate whether logical schema transformation improves Text-to-SQL performance across benchmarks, pipelines and LLM backbones. Our evaluation aims to answer these research questions:

- **RQ1:** Does logical schema transformation improve Text-to-SQL performance when historical workloads are available?
- **RQ2:** How do different operators behave across pipelines, and how do they compare with prior schema-level methods?
- **RQ3:** Are the gains robust across backbones, external knowledge, and templates unseen in the workload?
- **RQ4:** When does schema transformation help, when does it hurt, and can that be predicted before deployment?

6.1 Experimental Setup

6.1.1 Benchmarks.

- **BIRD-Union** merges all 11 BIRD [26] dev databases into one schema of 75 tables with 1,534 question–SQL pairs. Evidence and column descriptions are never utilized (provided *only* in Section 6.7). Diverse domains, ambiguous identifiers, and a large search space stress the importance of the schema.
- **Spider-Union** merges the Spider [43] dev databases into one schema of 78 tables with 1,004 pairs. Identifiers are cleaner, but schema linking over the enlarged schema is still a challenge.
- **BEAVER** [7] is an enterprise benchmark with three databases (dw, neutron, nova; 97, 175 and 109 tables). We use the full neutron and nova sets and sampled 1,000 pairs of the dw set for a total of 3070 pairs. This benchmark is hard with its wide tables, ambiguous identifiers and in-depth domain knowledge.

6.1.2 Pipelines, backbones, prior methods. We evaluate BASESQL, DIN-SQL [31], MAC-SQL [39], CSC-SQL [36] (Qwen2.5-Coder-7B on one A40, FP16), and the agentic schema-exploration pipeline AutoLink [40] (DeepSeek-V3 for linking, DeepSeek-R1 for generation). Backbones and temperature are held fixed between baseline and transformed variants. **Four prior schema-level methods are implemented to substitute the corresponding operator under BASESQL: SNAILS [27] (N_1 naturalisation, for +R), Agentic Schema Refinement [32] (LLM-generated views, for +A), EvoSchema’s [44] (only the merge-tables option as views, for +A), and TEMPLAR [3] (query-log keyword mapping and join-path inference, for +P).**

6.1.3 Schema transformations. **A** (abstraction), **P** (partitioning), and **R** (renaming) are evaluated alone and in every combination, and all three consume the same historical workload $\mathcal{H}$.

6.1.4 Metric and statistics. We report execution accuracy (EX), the standard metric for Text-to-SQL evaluation. For each pipeline and benchmark, we show the same pipeline against transformed-schema variants, and improvements are reported as absolute percentage point gains over the corresponding baseline. **Every p value is McNemar’s exact test on paired per-question correctness, and tables mark $p < 0.05$ with ** and $0.05 \leq p < 0.10$ with *.** Three independent runs of the full +A+P+R configuration across backbones and more in-depth p values are discussed in Section 6.8.

6.1.5 Historical workload setting. Question–SQL pairs are split evenly into a historical workload and a held-out test set (767/767 for BIRD-Union, 502/502 for Spider-Union, 1534/1536 for BEAVER due to rounding). The workload serves two roles: *design-time*, guiding schema transformation to apply the operators, and *prompt-time*, supplying top- k SQL demonstrations in the prompt. Baselines draw the demonstrations from the same pool, with schema transformation being the only difference, and test questions never enter the pool. The “50% Hist.” row sub-samples the historical workload to 50% (383 for BIRD-Union and 251 for Spider-Union), and this affects both the *design-time* and *prompt-time*, but the test set is unchanged.

6.2 Main Results With Historical Workloads

Table 2 report the transformation results. **Every pipeline and backbone improves under at least six of seven configurations, and every best result comes from a transformed schema.**

BIRD-Union BASESQL rises from 49.8 to 52.4, DIN-SQL from 44.2 to 48.2, MAC-SQL from 45.9 to 48.6 and CSC-SQL from 49.0 to 50.6. These improvements are meaningful because the baselines already exploit historical demonstration few-shot, so schema transformation provides complementary gains even on top of that. **The largest gains are on the agentic pipeline: AutoLink explores the schema itself, yet every configuration lifts it on BIRD-Union, by 2.7 to 5.7 points, all significant, and +A+R takes it from 51.5 to 57.2.**

Spider-Union BASESQL rises from 79.5 to 81.3, DIN-SQL from 74.9 to 79.1, MAC-SQL from 76.5 to 79.7, CSC-SQL from 82.7 to 84.5 and AutoLink from 79.9 to 82.5. The direction matches BIRD-Union, but with baselines already near 80% the room to gain is small.

The double-starred cells concentrate in the combined operators and the best operator depends on the LLM: GPT-4.1 mini and Gemini gain most from +A+P or +A+P+R, GPT-5.4 mini from +A alone. No single configuration dominates. This heterogeneity suggests that logical schema design should be treated as an adaptive optimization space, and the rest of the section asks which bottleneck each operator addresses, and when.

6.3 Analysis of Transformation Types

Schema abstraction (+A) is more effective when recurring join paths dominate the errors. By itself, it improves GPT-5.4 mini by 2.8 points on Spider-Union and 2.7 on BIRD-Union, and this gain is not merely a uniform improvement. On Spider-Union, JOIN-path errors account for 29.3% of the 92 baseline failures, and +A repairs these errors at a 33.3% rate, which is 44% higher than its repair rate on non-JOIN failures (23.1%), empirically confirming that abstraction specifically targets errors caused by complex join structures.

Schema partitioning (+P) changes both the schema the model sees and the pool it retrieves demonstrations from. Table 3 separates the two: pruning the schema alone is worth +1.3, scoping the demonstrations alone +0.0, and both together give +P’s +2.6 — scoping pays only once the demonstrations that reference pruned tables are gone. The benefit of +P is schema-side.

Schema renaming (+R) is more effective when identifier ambiguity is the dominant failure. It aids in finding the correct identifiers, such that in Spider-Union with Gemini, the table-set match percentage rises from 65% to 94%. The gain is not uniform, and it is limited when the model already knows the schema. **After restoring BIRD’s evidence and column descriptions, +R still gains, but +A+P would carry more of the combined improvement, since those can clear some ambiguities (Section 6.7).**

Table 2: History-aware results, top- $k=3$ demonstrations, GPT-4.1 mini unless noted. Deltas over the row’s baseline; *italic*: above baseline; grey: within ± 0.1 ; bold: best per row; **: McNemar $p < 0.05$, *: $0.05 \leq p < 0.10$. Same conventions in all tables.

Approach	Baseline	+A	+P	+R	+A+P	+A+R	+P+R	+A+P+R
BIRD-Union ($n=767$)								
BASeSQL	49.8	50.9 (+1.1)	50.5 (+0.7)	50.5 (+0.7)	52.4 (+2.6)**	51.4 (+1.6)	50.5 (+0.7)	52.3 (+2.5)*
BASeSQL (50% Hist.)	48.8	50.5 (+1.7)	51.0 (+2.2)*	49.9 (+1.1)	50.3 (+1.5)	50.6 (+1.8)	51.0 (+2.2)	51.9 (+3.1)**
DIN-SQL	44.2	45.2 (+1.0)	48.1 (+3.9)**	46.7 (+2.5)*	46.7 (+2.5)**	45.9 (+1.7)	48.2 (+4.0)**	48.2 (+4.0)**
MAC-SQL	45.9	47.2 (+1.3)	48.0 (+2.1)*	46.4 (+0.5)	48.5 (+2.6)**	48.5 (+2.6)*	48.4 (+2.5)*	48.6 (+2.7)**
CSC-SQL (Qwen2.5-Coder-7B)	49.0	49.4 (+0.4)	50.6 (+1.6)	50.3 (+1.3)	49.0 (0.0)	50.1 (+1.1)	49.9 (+0.9)	49.7 (+0.7)
AutoLink (DeepSeek-V3/R1)	51.5	57.1 (+5.6)**	56.3 (+4.8)**	55.8 (+4.3)**	54.2 (+2.7)**	57.2 (+5.7)**	56.1 (+4.6)**	56.2 (+4.7)**
<i>BASeSQL on other backbones</i>								
BASeSQL (Gemini 2.5-flash-lite)	48.1	48.0 (-0.1)	50.7 (+2.6)**	48.6 (+0.5)	51.6 (+3.5)**	50.6 (+2.5)*	51.4 (+3.3)**	52.3 (+4.2)**
BASeSQL (GPT-5.4 mini)	52.8	55.5 (+2.7)**	53.1 (+0.3)	52.9 (+0.1)	54.5 (+1.7)	54.5 (+1.7)	54.0 (+1.2)	53.9 (+1.1)
Spider-Union ($n=502$)								
BASeSQL	79.5	81.3 (+1.8)	79.5 (0.0)	80.1 (+0.6)	79.9 (+0.4)	79.7 (+0.2)	79.9 (+0.4)	81.3 (+1.8)
BASeSQL (50% Hist.)	77.9	79.5 (+1.6)*	77.1 (-0.8)	78.1 (+0.2)	78.1 (+0.2)	78.5 (+0.6)	78.9 (+1.0)	79.3 (+1.4)
DIN-SQL	74.9	76.3 (+1.4)	75.3 (+0.4)	77.5 (+2.6)*	76.5 (+1.6)	75.7 (+0.8)	79.1 (+4.2)**	77.5 (+2.6)*
MAC-SQL	76.5	77.5 (+1.0)	77.5 (+1.0)	78.9 (+2.4)*	77.5 (+1.0)	79.7 (+3.2)**	79.3 (+2.8)**	79.1 (+2.6)*
CSC-SQL (Qwen2.5-Coder-7B)	82.7	84.1 (+1.4)	83.1 (+0.4)	84.5 (+1.8)	84.1 (+1.4)	83.9 (+1.2)	83.1 (+0.4)	84.5 (+1.8)
AutoLink (DeepSeek-V3/R1)	79.9	82.5 (+2.6)**	80.9 (+1.0)	81.5 (+1.6)	80.9 (+1.0)	81.9 (+2.0)	81.5 (+1.6)	80.9 (+1.0)
<i>BASeSQL on other backbones</i>								
BASeSQL (Gemini 2.5-flash-lite)	80.5	80.5 (0.0)	81.7 (+1.2)	81.5 (+1.0)	81.5 (+1.0)	81.3 (+0.8)	82.9 (+2.4)	83.1 (+2.6)*
BASeSQL (GPT-5.4 mini)	81.7	84.5 (+2.8)**	82.5 (+0.8)	82.5 (+0.8)	82.3 (+0.6)	84.1 (+2.4)*	84.5 (+2.8)*	84.3 (+2.6)*

Table 3: Decomposing +P (BIRD-Union, BASeSQL, Gemini) to full/pruned schema $\times$ global/+P-scoped top- k

	Global top- k	Partition-scoped top- k
Full schema	48.1	48.1 (+0.0)
Pruned schema	49.4 (+1.3)	50.7 (+2.6)**

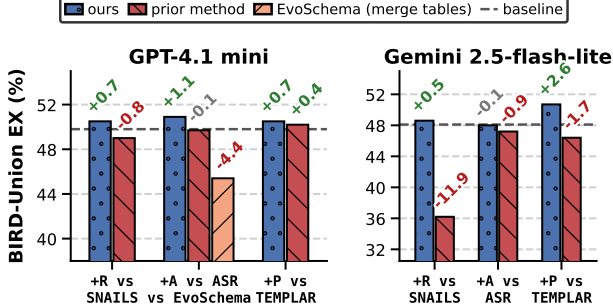


Figure 8: Prior schema-level methods vs. our operators on BIRD-Union (BASeSQL); labels: delta over baseline.

6.4 Comparison with Prior Schema-Level Methods

Each method is a prior work of one operator. Figure 8 compares them under BASeSQL on BIRD-Union with the GPT-4.1 mini and Gemini-2.5-flash-lite as backbones. The +A group carries one more bar of EvoSchema’s [44] merge-tables operation, its merged tables are considered views to compare against +A.

SNAILS falls below the baseline on both backbones and TEMPLAR stays at it (+0.4% and -1.7%), whereas their respective operators, +R and +P, are above the baseline on both, and Agentic Schema Refinement (ASR) is all slightly below (-0.1% and -0.9%). SNAILS lengthens identifiers without making them necessarily correct and only renames the non- N_1 s, and this caused Gemini to fall back to the names it memorized at -11.9% (Section 6.6). ASR creates

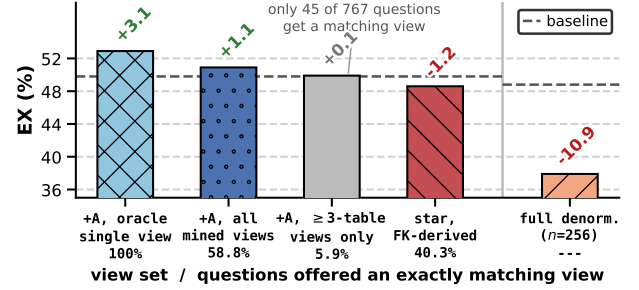


Figure 9: Abstractions on BIRD-Union (BASeSQL, GPT-4.1 mini). Denormalization restricted to its 256 expressible ones.

825 views (vs 64 views from +A) on the 75 base tables, this adds noises for SQL generation while also taking up more tokens. The average views injected per question for ASR is 35.8 while it is only 4.4 for +A, and it lengthens the average token needed per question on BASeSQL by 20.5% and 8.8% more than +A. TEMPLAR infers a join path per question from the log, and its recall is low on the 75-table schema. Its output contains every gold table only 46.2% of the time, and 67.5% of its paths join tables from different databases on BIRD-Union, whereas +P contains the gold table 96.9% of the time while averaging only 5.3 tables. EvoSchema’s merge-tables operation merges related tables into wider ones, and this improves table selection over the baseline (83.8% vs 80.2% of questions with the exact table set). But the EX still falls (45.4% vs 49.8%), and over half of its new errors (43 of 81) are projecting wrong columns.

6.5 What Makes an Abstraction Useful

Figure 9 compares five ways of constructing the +A. One important metric is that how often does any of the retrieved views matches exactly the question’s tables. The star set only has 40% match rate, so it results in a degradation. The +A set has a 58.8% to result in a slight gain, and the oracle (100%) gains most. The scenario with only views of three-or-more-tables is the exception because it only offers a matching view on 5.9% questions, so it is minimal impact.

However, even with ground-truth retrieval on the +A set, the match rate caps out at 60.4% because not all table sets has a corresponding view. If we isolate the questions where the correct view is offered, the +A+P configuration has the highest gain of 4.5% for that part.

Star schema. Star-schema, commonly used in data marts, is the obvious alternative to the current +A, so we built the star-schema only from the FK relationships. The number of views is on par with 46 vs 64 and the coverage is even better such that it can express more of the gold queries (79.5% vs 67.1%). However, it still loses to +A and even falls below the baseline. The reason is *over-joining*. The average number of tables per view for +A set is 2.3, while star-schema is 2.7. For each question, when we take the best view offered and count the tables in it: star-schema carries 0.86 extra tables on average, whereas +A set is only 0.34. The FK graph says what *can* be joined, and the workload says what *is* joined together.

Denormalization. This is similar to the universal relation of classical database theory [28], and fuses each database into one relation. Denormalization has a few innate drawbacks: rows with no join partner are lost, missing matches are padded with NULLs [33], and one-to-many joins duplicate rows. We build each relation with LEFT JOINs outward from one base table, to keep its granularity. This is the construction a data mart uses for its fact table [21]. In the end, only 256 out of 767 questions are evaluated because only they are expressible by using the denormalized views, and plummets EX from 48.8% to 37.9%. The largest component drop is on selection (WHERE-column match falls by 27.5%). 62% of the questions that was correct but wrong after the denormalization has a structurally correct query but return the wrong rows due to duplication, NULL padding, or base table’s granularity.

6.6 Validating the Renaming Operator

SNAILS is below the baseline is because its naturalness objective rewards longer names, not correct ones. In BIRD-Union’s schema, it expands the betting codes into wrong meanings, such as BWD (Bet&Win draw odds) becomes bandwidth, B365A (Bet365 away odds) becomes bet365_account. With +R, its goal is not solely for expansion, but it is clearing ambiguity from the past workload. Another drawback of SNAILS is that it only renames all non- N_1 (not at max naturalness), so it leaves 73 of 75 tables and 534 of 798 columns unchanged. This is a smaller problem for the GPT-4.1 mini (-0.8%), but it was a huge problem for the Gemini (-11.9%). Gemini was hallucinating the original column name instead of the newly renamed name because only partial of the schema is renamed. Another check to show that +R’s renames are correct rather than just different. On the columns from the schema linking stage, embedding cosine similarity distance to the question rises from 0.69 to 0.712 (+0.022) under +R and to 0.696 (+0.006 under SNAILS).

Human validation. We manually inspect the generated renames and found three incorrect column renames, all under one database. We correct them and rerun +R on the 82 questions for that database. EX changes by -2.44 points on GPT-4.1 mini, -4.88 on GPT-5.4 mini, and +6.10 on Gemini. Correcting these errors therefore does not consistently improve +R, although the effect depends on the backbone.

6.7 Robustness

With external knowledge. Adding back BIRD’s evidence and column descriptions results in a gain of 11.4% on GPT-4.1 mini and

Table 4: BIRD-Union with evidence & descriptions, BASESQL.

Backbone	Baseline	+R	+A+P	+A+P+R
GPT-4.1 mini	61.2	62.7 (+1.6)	62.8 (+1.7)	63.2 (+2.1)*
Gemini 2.5-flash-lite	58.8	59.3 (+0.5)	62.8 (+4.0)**	62.7 (+3.9)**

Table 5: Template-disjoint subsets of BIRD-Union, BASESQL.

Test set	Baseline	+A	+P	+A+P
skeleton-disjoint ($n=599$)	45.2	46.4 (+1.2)	46.2 (+1.0)	47.4 (+2.2)
keyword-disjoint ($n=497$)	44.3	44.9 (+0.6)	45.1 (+0.8)	46.3 (+2.0)

10.7% on Gemini(Table 4). Even with additional information, the transformations still have a decent gain: +R gains as much as before or more (1.6% and 0.5%), +A+P keeps most of its gain for GPT-4.1 mini(1.7%) and is the strongest configuration for Gemini at 4.0%, and the full +A+P+R remains the best for GPT-4.1 mini. External knowledge and the transformations are complementary rather than substitutes such as the description may disambiguate the columns, but it cannot tell the model which join path to take or how to write the selection clause, and those are what the transformations supply.

Across workload availability. Putting the historical workload to 50% (the “50% Hist.” rows) costs the baseline 1% since the *prompt-time* pool is halved. At the same time, the *design-time* pool is also halved, it would create less views and partitions, renaming would be more difficult. Every BIRD-Union configuration still improves, with +A+P+R improves the most by 3.1. On Spider-Union, six of seven configurations improve, +P is heavily affected because of how much split the partition are (251 history across 19 databases). In the zero-shot setting on BIRD-Union, BASESQL with GPT-4.1 mini reaches 44.7%, and the best configuration, +A, reaches 45.0%. The full results are in the supplementary material in the link.

Distribution shift. The history/test split is random, so we test if the gains depend on a test question having a similar query in the history. A test question is evaluated only if no history has the *same tables* with the *same shape*, and the two disjoints only differ in the definition of the *same shape*. For skeleton-disjoint, the SQL has its literals and identifiers masked, and rest is compared for the shape. As for keyword-disjoint, the set of SQL keywords from the query is its shape, so it is coarser. These questions are generally harder, so the baseline on BASESQL (GPT-4.1 mini) drops 4.6% and 5.5% (Table 5). But each configuration still have a gains, the combined configuration (+A+P) stay strong with the most gain of over 2%.

6.8 Statistical Reliability

Every cell carries McNemar’s exact test against its row’s baseline, and ** marks $p < 0.05$ and * marks $0.05 \leq p < 0.10$. In Table 2, 25 cells reach $p < 0.05$ and another 16 fall between 0.05 and 0.10, and most of they concentrate in the combined configurations and in AutoLink. McNemar’s test looks only at the questions on which two systems disagree, for example a pipeline’s configuration against the baseline of that row, and reports as the p -value how likely a split that lopsided (i.e., 74 vs 38 for Autolink shown in Figure 10) would be if each disagreement were decided by a coin flip [29]. It is the standard paired test for comparing two classifiers evaluated on the same examples [12, 13]. As for repeated runs, three runs of +A+P+R are performed on each backbone and benchmark to reach a standard deviation of 0.2% to 0.8%, and largest being GPT-5.4 mini on BIRD-Union (54.2 ± 0.8) and smallest being Gemini (52.5 ± 0.2).

6.9 Enterprise Schemas: BEAVER

To test the effectiveness of the operators beyond the union benchmarks, we turn to BEAVER, an enterprise benchmark with wide tables, ambiguous identifiers, and complex SQL structures, with three databases pool 1,536 test questions (Table 6). The pattern repeats: combined configurations are almost always better, no single operator exceeds 0.7%, while the combined configurations reach up to 1.6%. dw’s identifiers are the most ambiguous, so +R matters the most and +A+R gives the most gain for that database. neutron is the widest at 175 tables, so +P is really effective and +P+R provides the most gain. Nova’s identifiers are more descriptive, but it still needs +A+P to filter out the unnecessary tables(109 tables), which results in the most gain. Across all three databases, +P+R cuts the prompt by 56–75% while having a positive impact on the final EX.

Table 6: Results on BEAVER, an enterprise benchmark, with $n=1,536$, BASESQL, GPT-4.1 mini, and top- $k=10$.

	Baseline	+A	+P	+R	+A+P	+A+R	+P+R	+A+P+R
EX	21.3	21.7 (+0.5)	22.0 (+0.7)	21.5 (+0.3)**	22.9 (+1.6)*	22.7 (+1.4)	22.4 (+1.1)*	22.8 (+1.5)

6.10 Cost

+A and +R creates only logical views, so neither adds materialized data. Offline, one full transformation of BIRD-Union costs 0.67M input tokens and 0.19M output tokens (including all renaming, view DDLs, and rewriting the history). When online, +R is free, +A adds about 11%, +P removes 56%, and the full +A+P+R cuts 45% prompt tokens than the baseline. The full results are in the supplementary material in the link.

6.11 Error Analysis and Boundary Conditions

Figure 10 scores two pipelines’ output on the questions it gains and the questions it loses. For each of the six categories, it scores the correctness of that category on the questions it gained minus the questions it lost. For example, for BASESQL with Gemini (+A+P+R, 74 gained, 42 lost), the literal matching for the 42 lost ones is 71.5%, and it is 73% for the 74 gained, so the difference is only 1.5% (shown in the leftmost category with the blue bar). Therefore, focusing on Gemini ones (blue bars), most losses were structural. On literals, WHERE columns and aggregates the two subsets differ by at most 9%, whereas on tables, join conditions and projections by 22.6% to 27.3%. For AutoLink (+A+P+R, 74 gained, 38 lost) the largest gap moves to the fine-grained side, such that lost questions still get the SELECT columns right 47.4%(vs 54.1%) of the time, but WHERE columns fall to 26.3%(vs 47.3%). This results in a up to 21% gap on the more fine-grained components against up to 13% in the structural part.

Error propagation. +R is a one-to-one name mapping and removes no table or column, so its main failure mode is incorrect semantic enrichment rather than loss of schema information that later operators inherit; Section 6.6 examines these cases. A more direct propagation risk occurs from +P into +A: +P omits at least one gold table on 24 of 767 questions (3.1%), restricting the information available to subsequent operators. However, we do not observe evidence that adding +A systematically amplifies this error. Compared with +P, +A+P gains 29 questions and loses 14 on GPT-4.1 mini; even among the 24 questions with a missing gold table, +A+P answers 4 correctly compared with 3 for +P. The upstream error itself is nevertheless costly: when the partition contains all gold

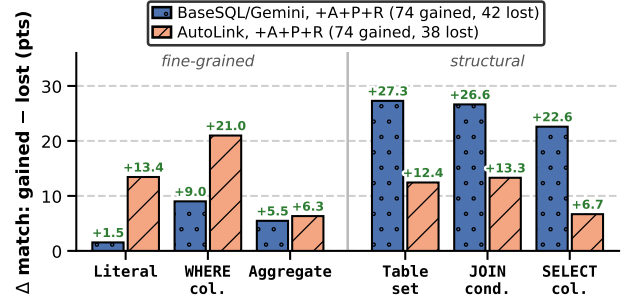


Figure 10: Where the losses come from: each pipeline’s component-match accuracy on the questions it gains minus on those it loses (BIRD-Union).

tables, +P and +A+P achieve 51.7% and 53.6% EX, respectively, compared with 12.5% and 16.7% when a gold table is missing. Thus, the main robustness requirement is high recall in the upstream schema-localization step: once a required table has been pruned from the model-facing schema, later operators generally cannot restore that missing schema information.

When each transformation applies. Returning to RQ4, each operator is more effective when it supplies something the schema lacks, and there are hints on how each condition can be read off the schema and the workload. +A is more effective when join patterns recur in the historical workload, so the frequently used join paths can be offered as views. +P is useful on wide schemas, and it is why the best single operator on the widest schema, nova, and part of the best combination on every BEAVER database. +R aims at identifiers that are ambiguous, and it is dw’s best configuration, but it can be less effective when additional information is provided. None of this depends on the pipeline: for AutoLink, which performs explicit schema exploration, it has the most gain over any other pipelines.

7 CONCLUSION AND FUTURE WORK

In this paper, we introduced Text-to-SQL-friendly logical database design, a **design objective** that shifts the reasoning burden from the language model’s prompt to the underlying schema representation. By achieving this objective through three transformations—Schema Abstraction (+A), Schema Partitioning (+P), and Schema Renaming (+R)—we demonstrated that optimizing the model-facing schema yields pipeline-agnostic performance gains. Our framework seamlessly integrates with existing Text-to-SQL systems, delivering consistent accuracy improvements across multiple LLM backbones in both zero-shot and few-shot settings. Ultimately, as the primary consumer of the schema shifts from human developers to language models, Text-to-SQL-friendly logical design will become just as fundamental an objective as classical normalization and data integrity.

Moving forward, there are several promising avenues for extending this work. First, future frameworks could focus on dynamic, cost-aware view materialization for massive-scale databases. Second, subsequent research could explore context-aware schema renaming tailored to the user’s specific phrasing. Finally, developing adaptive routing mechanisms will be critical to mitigate domain-specific regressions.

REFERENCES

- [1] Sanjay Agrawal, Surajit Chaudhuri, and Vivek R. Narasayya. 2000. Automated Selection of Materialized Views and Indexes in SQL Databases. In *Proceedings of the 26th International Conference on Very Large Data Bases (VLDB)*. VLDB Endowment, 496–505. <http://www.vldb.org/conf/2000/P496.pdf>
- [2] Sanjay Agrawal, Vivek R. Narasayya, and Beverly Yang. 2004. Integrating Vertical and Horizontal Partitioning Into Automated Physical Database Design. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. ACM, 359–370. <https://doi.org/10.1145/1007568.1007609>
- [3] Christopher Baik, H. V. Jagadish, and Yunyao Li. 2019. Bridging the Semantic Gap with SQL Query Logs in Natural Language Interfaces to Databases. In *35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8–11, 2019*. IEEE, 374–385. <https://doi.org/10.1109/ICDE.2019.00041>
- [4] Zhenbiao Cao, Yuanlei Zheng, Zhihao Fan, Xiaojin Zhang, Wei Chen, and Xiang Bai. 2024. RSL-SQL: Robust Schema Linking in Text-to-SQL Generation. *arXiv preprint arXiv:2411.00073* (2024). <https://doi.org/10.48550/arXiv.2411.00073>
- [5] Surajit Chaudhuri and Vivek Narasayya. 1997. An Efficient, Cost-Driven Index Selection Tool for Microsoft SQL Server. In *Proceedings of the 23rd International Conference on Very Large Data Bases (VLDB)*.
- [6] Surajit Chaudhuri and Gerhard Weikum. 2008. Foundations of Automated Database Tuning. *Proceedings of the VLDB Endowment* 1, 1 (2008), 3–14.
- [7] Peter Baile Chen, Devin Yang, Weiyue Li, Fabian Wenz, Yi Zhang, Nesime Tatbul, Michael Cafarella, Çağatay Demiralp, and Michael Stonebraker. 2026. BEAVER: An Enterprise Benchmark for Text-to-SQL. *arXiv:2409.02038 [cs.CL]* <https://arxiv.org/abs/2409.02038>
- [8] b-mc2 Contributors. 2022. SQL Create Context: A Dataset with Attribute-Annotated DDL for Text-to-SQL. Hugging Face dataset. <https://huggingface.co/datasets/b-mc2/sql-create-context>
- [9] Carlo Curino, Yang Zhang, Evan P. C. Jones, and Samuel Madden. 2010. Schism: a Workload-Driven Approach to Database Replication and Partitioning. *Proceedings of the VLDB Endowment* 3, 1 (2010), 48–57. <https://doi.org/10.14778/1920841.1920853>
- [10] Databricks. 2025. Semantic Layer Architecture: Components, Design Patterns and AI Integration. Databricks Blog. <https://www.databricks.com/blog/semantic-layer-architecture-components-design-patterns-and-ai-integration>
- [11] Minghang Deng, Ashwin Ramachandran, Canwen Xu, Lanxiang Hu, Zhewei Yao, Anupam Datta, and Hao Zhang. 2025. ReFoRCE: A Text-to-SQL Agent with Self-Refinement, Format Restriction, and Column Exploration. In *ICLR 2025 Workshop on VeriFAI*.
- [12] Thomas G. Dietterich. 1998. Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms. *Neural Comput.* 10, 7 (1998), 1895–1923. <https://doi.org/10.1162/089976698300017197>
- [13] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. The Hitchhiker’s Guide to Testing Statistical Significance in Natural Language Processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, ACL 2018*. Association for Computational Linguistics, 1383–1392. <https://doi.org/10.18653/V1/P18-1128>
- [14] Jeffrey Eben, Aitzaz Ahmad, and Stephen Lau. 2025. RASL: Retrieval Augmented Schema Linking for Massive Database Text-to-SQL. *CoRR* abs/2507.23104 (2025). <https://doi.org/10.48550/ARXIV.2507.23104> *arXiv:2507.23104*
- [15] Jonathan Fürst, Catherine Kosten, Farhad Nooralahzadeh, Yi Zhang, and Kurt Stockinger. 2025. Evaluating the Data Model Robustness of Text-to-SQL Systems Based on Real User Queries. In *Proceedings of the 28th International Conference on Extending Database Technology (EDBT)*.
- [16] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proceedings of the VLDB Endowment* 17, 5 (2024), 1132–1145.
- [17] Hector Garcia-Molina, Jeffrey D. Ullman, and Jennifer Widom. 2008. *Database Systems: The Complete Book* (2 ed.). Pearson.
- [18] Alon Y. Halevy. 2001. Answering queries using views: A survey. *The VLDB Journal* 10 (2001), 270–294. <https://doi.org/10.1007/s007780100054>
- [19] Venky Harinarayan, Anand Rajaraman, and Jeffrey D. Ullman. 1996. Implementing Data Cubes Efficiently. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. ACM Press, 205–216. <https://doi.org/10.1145/233269.233333>
- [20] Alekh Jindal, Konstantinos Karanasos, Sriram Rao, and Hiren Patel. 2018. Selecting subexpressions to materialize at datacenter scale. *Proceedings of the VLDB Endowment* 11, 7 (2018), 800–812.
- [21] Ralph Kimball and Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). John Wiley & Sons.
- [22] Yannis Kotidis and Nick Roussopoulos. 1999. DynaMat: A Dynamic View Management System for Data Warehouses. In *Proceedings of the 1999 ACM SIGMOD International Conference on Management of Data*. 371–382.
- [23] Boyan Li, Chong Chen, ZhuJun Xue, Yinan Mei, and Yuyu Luo. 2026. DeepEye-SQL: A Software-Engineering-Inspired Text-to-SQL Framework. *Proceedings of the ACM on Management of Data*.
- [24] Boyan Li, Jiayi Zhang, Ju Fan, Yanwei Xu, Chong Chen, Tang Nan, and Yuyu Luo. 2025. Alpha-SQL: Zero-Shot Text-to-SQL using Monte Carlo Tree Search. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*.
- [25] Haoyang Li, Shang Wu, Xiaokang Zhang, Xinmei Huang, Jing Zhang, Fuxin Jiang, Shuai Wang, Tieying Zhang, Jianjun Chen, Rui Shi, Hong Chen, and Cuiping Li. 2025. OmniSQL: Synthesizing High-Quality Text-to-SQL Data at Scale. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4695–4709. <https://doi.org/10.14778/3749646.3749723>
- [26] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.
- [27] Kyle Luoma and Arun Kumar. 2025. SNAILS: Schema Naming Assessments for Improved LLM-Based SQL Inference. *Proceedings of the ACM on Management of Data (PACMOD)* 3, 1, Article 77 (February 2025), 26 pages. <https://doi.org/10.1145/3709727>
- [28] David Maier, Jeffrey D. Ullman, and Moshe Y. Vardi. 1984. On the Foundations of the Universal Relation Model. *ACM Trans. Database Syst.* 9, 2 (1984), 283–308. <https://doi.org/10.1145/329.318580>
- [29] Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12, 2 (1947), 153–157.
- [30] Andrew Pavlo, Gustavo Angulo, Joy Arulraj, Haibin Lin, Jiexi Lin, Lin Ma, Prashanth Menon, Todd C. Mowry, Matthew Perron, Ian Quah, Siddharth Santurkar, Anthony Tomasic, Skye Toor, Dana Van Aken, Ziqi Wang, Yingjun Wu, Ran Xian, and Tieying Zhang. 2017. Self-Driving Database Management Systems. In *Proceedings of the 8th Biennial Conference on Innovative Data Systems Research (CIDR)*. <http://cidrdb.org/cidr2017/papers/p42-pavlo-cidr17.pdf>
- [31] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36. 28448–28472.
- [32] Agapi Rissaki, Ilias Fountalis, Nikolaos Vasiloglou, and Wolfgang Gatterbauer. 2024. Towards Agentic Schema Refinement. In *3th Table Representation Learning Workshop Advances at Neural Information Processing Systems (NeurIPS)*. <https://openreview.net/forum?id=JzYEHYomME>
- [33] Yehoshua Sagiv. 1981. Can We Use the Universal Instance Assumption Without Using Nulls?. In *Proceedings of the 1981 ACM SIGMOD International Conference on Management of Data*. ACM Press, 108–120. <https://doi.org/10.1145/582318.582336>
- [34] Diptikalyan Saha, Avriela Floratou, Karthik Sankaranarayanan, Umar Farooq Minhas, Ashish R Mittal, and Fatma Özcan. 2016. ATHENA: an ontology-driven system for natural language querying over relational data stores. *Proceedings of the VLDB Endowment* 9, 12 (2016), 1209–1220.
- [35] Jaydeep Sen, Chuan Lei, Abdul Quamar, Fatma Özcan, Vasilis Efthymiou, Ayushi Dalmia, Greg Stager, Ashish R. Mittal, Diptikalyan Saha, and Karthik Sankaranarayanan. 2020. ATHENA++: Natural Language Querying for Complex Nested SQL Queries. *Proceedings of the VLDB Endowment* 13, 11 (2020), 2747–2759.
- [36] Lei Sheng and Xu Shuai Shuai. 2025. CSC-SQL: Corrective Self-Consistency in Text-to-SQL via Reinforcement Learning. In *Findings of the Association for Computational Linguistics: IJCNLP-AAACL 2025*. Association for Computational Linguistics, 1473–1496. <https://doi.org/10.18653/v1/2025.findings-ijcnlp.91>
- [37] Shayan Taleai, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. 2025. CHES: Contextual Harnessing for Efficient SQL Synthesis. In *ICML 2025 Workshop on Multi-Agent Systems in the Era of Foundation Models*.
- [38] Kapil Vaidya, Jialin Ding, Sebastian Kosak, David Kernert, Chuan Lei, Xiao Qin, Abhinav Tripathy, Ramesh Balan, Balakrishnan Narayanaswamy, and Tim Kraska. 2025. TailorSQL: A NL2SQL System Tailored for Your Query Workload. In *Proceedings of Workshops at the 51st International Conference on Very Large Data Bases, VLDB 2025, London, UK, September 1–5, 2025*. VLDB Endowment. https://www.vldb.org/2025/Workshops/VLDB-Workshops-2025/AIDB/AIDB25_2.pdf
- [39] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Linzheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, et al. 2025. MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. In *Proceedings of the 31st International Conference on Computational Linguistics*. 540–557.
- [40] Ziyang Wang, Yuanlei Zheng, Zhenbiao Cao, Xiaojin Zhang, Zhongyu Wei, Pei Fu, Zhenbo Luo, Wei Chen, and Xiang Bai. 2026. AutoLink: Autonomous Schema Exploration and Expansion for Scalable Schema Linking in Text-to-SQL at Scale. In *Fortieth AAAI Conference on Artificial Intelligence, Thirty-Eighth Conference on Innovative Applications of Artificial Intelligence, Sixteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2026, Singapore, January 20–27, 2026*. AAAI Press, 33809–33817. <https://doi.org/10.1609/AAAI.V40I40.40672>
- [41] Xiangjin Xie, Guangwei Xu, Lingyan Zhao, and Ruijie Guo. 2025. OpenSearch-SQL: Enhancing Text-to-SQL with Dynamic Few-shot and Consistency Alignment. *Proceedings of the ACM on Management of Data* 3, 3, Article 194 (2025), 24 pages. <https://doi.org/10.1145/3725331>
- [42] Wenbo Xu, Haifeng Zhu, Liang Yan, Chuanyi Liu, Peiyi Han, Shaoming Duan, and Jeff Z. Pan. 2025. TS-SQL: Test-driven Self-refinement for Text-to-SQL. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association

for Computational Linguistics, 2864–2889.

- [43] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*.
- [44] Tianshu Zhang, Kun Qian, Siddhartha Sahai, Yuan Tian, Shaddy Garg, Huan Sun, and Yunyao Li. 2025. EvoSchema: Towards Text-to-SQL Robustness Against Schema Evolution. *Proceedings of the VLDB Endowment* 18, 10 (2025), 3655–3668.
- [45] Chen Zhao, Yu Su, Adam Pauls, and Emmanouil Antonios Platanios. 2022. Bridging the Generalization Gap in Text-to-SQL Parsing with Schema Expansion. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*. 5568–5578. <https://doi.org/10.18653/v1/2022.acl-long.381>
- [46] Jingren Zhou, Per-Ake Larson, Johann-Christoph Freytag, and Wolfgang Lehner. 2007. Efficient exploitation of similar subexpressions for query processing. In *Proceedings of the 2007 ACM SIGMOD international conference on Management of data*. 533–544.
- [47] Daniel C. Zilio, Jun Rao, Sam Lightstone, Guy M. Lohman, Adam J. Storm, Christian Garcia-Arellano, and Scott Fadden. 2004. DB2 Design Advisor: Integrated Automatic Physical Database Design. In *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*. VLDB Endowment, 1087–1097. <https://doi.org/10.1016/B978-012088469-8.50095-4>